PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination

submitted in partial fulfilment of the requirement
for the award of the Degree of

Bachelor of Technology

in

Computer Engineering

by

Yateen Vaviya (2023300251)

Vinayak Yadav (2023300264)

Sahil Gupta (2023300072)

under the guidance of

Dr. Rupali Sawant



Department of Computer Engineering
Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri (W), Mumbai-400058
University of Mumbai

Certificate

This is to certify that the Project entitled "PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination" has been completed to our satisfaction by Yateen Vaviya, Vinayak Yadav and Sahil Gupta under the guidance of Dr. Rupali Sawant for the award of Degree of Bachelor of Technology in Computer Engineering from University of Mumbai.

Certified by

Dr. Rupali Sawant
Project Guide

Dr. Sudhir Dhage
Head of Department

Dr. Bhalchandra Chaudhari
Principal



Department of Computer Engineering
Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri (W), Mumbai-400058
University of Mumbai

Project Approval Certificate

This is to certify that the Project entitled "PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination" by Yateen Vaviya, Vinayak Yadav and Sahil Gupta is found to be satisfactory and is approved for the award of Degree of Bachelor of Technology in Computer Engineering from University of Mumbai.

External Examiner

(signature)

Name:

Date:

Internal Examiner

(signature)

Name:

Date:

Seal of the Institute

Statement by the Candidates

We wish to state that the work embodied in this thesis titled "PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination" forms our own contribution to the work carried out under the guidance of Dr. Rupali Sawant at the Sardar Patel Institute of Technology. We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission.

Name and Signature:

1. Yateen Vaviya (2023300251)

2. Vinayak Yadav (2023300264)

3. Sahil Gupta (2023300072)

Acknowledgements

We express our sincere gratitude to our guide, Dr. Rupali Sawant, for the guidance and encouragement given throughout this project, and for the freedom we were given in shaping it. We are equally grateful to our mentor, Prof. Dayanand Ambawade, for the advice and support given throughout the work.

We thank our examiners for their advice, and Dr. Sudhir Dhage, Head of the Department of Computer Engineering, and its staff for the facilities required to carry out this work.

We thank our families and well-wishers for their constant encouragement.

Contents

List of Abbreviations **x**

Abstract **xi**

1 Introduction **1**

1.1 Problem Statement 1
1.2 Objectives 2
1.3 Scope 3
1.4 Technologies Used 3
1.5 Assumptions and Constraints 4
1.6 Layout of the Report 5

2 Literature Survey **6**

3 Design **12**

3.1 System Architecture 12
3.2 Data Flow Diagram 12
Level 0 — Context Diagram 12
Level 1 — System Decomposition 15
3.3 Use Case Diagram 16
3.4 Sequence Diagram 16

4 Implementation **19**

4.1 Node Discovery and the Address Book 19
4.2 Joining Across Networks 19
4.3 Security and Identity 22
4.4 Placement and Scoring 23
4.5 The Lease Protocol 25
4.6 Reproducing the Environment 26
4.7 Closure Transfer and Cache Reuse 27
4.8 Execution and Isolation 28
4.9 Transparent Interception 30
4.9.1 Parallel Maps 31

4.9.2 Data-Parallel Training 32
4.10 The Cost Model 36
4.11 Fault Tolerance 37
4.12 Resource Estimation and Run Reporting 38

5 Results **39**
5.1 Evaluation Setup 39
5.2 Functional Verification 40
5.3 Environment Cache 40
5.4 Interception and Sandbox Overheads 41
5.5 Distribution Across Machines 42
5.6 Training 43
5.7 Fault Tolerance and Recovery 45
5.8 End-to-End Runs in the LAN Lab 45
5.9 Measurements Between Real Machines 46
5.10 Comparative Analysis 48

6 Conclusion **49**
6.1 Conclusion 49
6.2 Limitations 49

7 Future Scope **51**

8 Research Publication **52**
8.1 PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination 52

Bibliography **53**

A Implementation Size and Further Measurements **56**
A.1 Scale of the Implementation 56
A.2 Further Measurements 56

List of Figures

3.1 PipedPeer system design 13
3.2 Level 0 DFD (context diagram) 13
3.3 Level 1 DFD 15
3.4 Use case diagram 16
3.5 Sequence diagram of one job 17

4.1 Joining across networks 20
4.2 The three layers of trust 22
4.3 Placement flow 24
4.4 Lease states 25
4.5 Closure cache and transfer 27
4.6 The sandbox 28
4.7 Shim injection 30
4.8 Hash shuffle 33
4.9 Training synchronisation over the daemon port 35
4.10 Cost model decision regions 37
4.11 Racing local and remote work 38

5.1 Cold and warm submissions 41
5.2 Cost of interception 42
5.3 Sandbox start cost 42
5.4 Exact and fast synchronisation 44

List of Tables

1.1 Technologies used in PipedPeer 4

2.1 Summary of the surveyed work 10

3.1 Subsystems and their responsibilities 14

4.1 Intercepted primitives and their distribution strategies 32

5.1 Two-rank ring at the default interval 44
5.2 Failure drills 45
5.3 End-to-end steps, first run and cached 46
5.4 Two laptops over the internet (RTX 3060 and RTX 3050) 46
5.5 Further measurements between real machines 47
5.6 PipedPeer against existing systems 48

A.1 Implementation size 56
A.2 Further measurements 56

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
BLAS	Basic Linear Algebra Subprograms
BOINC	Berkeley Open Infrastructure for Network Computing
CLI	Command-Line Interface
CPU	Central Processing Unit
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture
D2D	Device-to-Device
DDP	Distributed Data Parallel (PyTorch)
DFD	Data Flow Diagram
DiLoCo	Distributed Low-Communication training
EDA	Event-Driven Architecture
FLOP	Floating-Point Operation
GFLOP	Giga (billion) Floating-Point Operations
GPU	Graphics Processing Unit
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
IGD	Internet Gateway Device
IoT	Internet of Things
IP	Internet Protocol
IPC	Inter-Process Communication
JSON	JavaScript Object Notation
LAN	Local Area Network
MEC	Multi-access Edge Computing
MLP	Multi-Layer Perceptron
MQTT	Message Queuing Telemetry Transport
NAT	Network Address Translation
NAT-PMP	NAT Port Mapping Protocol
NCCL	NVIDIA Collective Communications Library
OCI	Open Container Initiative
OTP	Open Telecom Platform
PCP	Port Control Protocol
POSIX	Portable Operating System Interface
QUIC	QUIC transport protocol (originally Quick UDP Internet Connections)
REST	Representational State Transfer
RFC	Request for Comments
ROCm	Radeon Open Compute platform
RTT	Round-Trip Time
SGD	Stochastic Gradient Descent
SQLite	SQL (Structured Query Language) database library
STUN	Session Traversal Utilities for NAT
SVD	Singular Value Decomposition
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TUI	Text-based User Interface
UDP	User Datagram Protocol
UPnP	Universal Plug and Play
URL	Uniform Resource Locator
VM	Virtual Machine
VPN	Virtual Private Network
VRAM	Video Random-Access Memory

Abstract

Idle computers are common in homes, offices and college laboratories, but using them for a Python script normally requires a configured cluster, a program rewritten against a cluster API, and machines that can reach one another through their routers. This report presents **PipedPeer**, a system that runs an unmodified Python script on other machines without a configured cluster, a rewritten program or an open route between the machines. Each machine runs one daemon from a single binary; the submitting machine builds the script's environment as a pinned **Nix closure**, places the job by live capacity, sends only the store paths the chosen machine lacks, and runs the job in a `crun` sandbox under its own memory ceiling. Placement runs in the submitting client under a lease, and every distributed path falls back to a local one. A shim injected into the interpreter **intercepts the parallel primitives the script already uses** — `multiprocessing.Pool`, `joblib`, pandas, NumPy and torch data-parallel training — and spreads their work across machines in proportion to measured throughput, only when a cost model predicts a gain. Machines on different networks meet through an introducer that every daemon provides, then connect directly over QUIC links, authenticated by each node's key and a shared cluster secret, after port mapping or hole punching; no relay carries job data and no central server exists. NAT traversal was evaluated on a two-site container testbed; the runs between physical machines in different homes used Tailscale. An Android phone under Termux joins as a worker for parallel chunks.

In a container lab, a job whose environment is already in place starts in a median of 0.83 s against 23.4 s for its first submission. Between two laptops in different homes, joined over Tailscale, a mixed CPU script finished in about 14 s against 18 s on one laptop. On a two-rank training ring, synchronising every 32 steps instead of every step kept the validation loss no more than 1% above one device's on every test model, with all ranks in one process (mean of three seeds), and in a separate container-lab run (one seed) each rank sent 7.6 to 9.2 times fewer bytes. Each of 95 features was broken in turn; the suite caught every mutation.

Keywords: Distributed Computing, Peer-to-Peer, NAT Traversal, Transparent Interception, Nix, Content-Addressed Caching, OCI Sandboxing, Cost-Based Scheduling, Data-Parallel Training, Fault Tolerance.

Chapter 1

Introduction

Much everyday computation is already parallel. A parameter sweep, a cross-validated model search, a simulation over many seeds or the training of a small network is written with `multiprocessing.Pool`, `joblib` or PyTorch, and runs on one machine because that is where it was written, while other machines nearby sit idle: the laptops of a project group, the workstations of a college laboratory after hours, a desktop at home. Using them is rarely worth the setup, so the work waits instead. This project sets out to make that choice free: a script uses other machines when that is faster, and runs as before when it is not.

1.1 Problem Statement

A developer with a laptop, a laboratory workstation and a spare desktop has a considerable amount of idle computing capacity and no practical way to use it. The frameworks that exist for distributed computation — Apache Spark [1], Ray [2], Dask [3] — presuppose a cluster that somebody has already configured: a head node, matching software on every worker, a scheduler process, and a network on which every machine can reach every other. Ordinary machines fail the last of these, since each sits behind its own home or campus router and accepts no inbound connection. The alternative, `ssh` with `rsync`, means reproducing the environment by hand on every machine and receiving no scheduling, isolation or fault tolerance in return.

Three distinct problems sit underneath this:

1. **The environment problem.** A script needs a particular interpreter and a particular set of libraries, and making a remote machine match the local one is the bulk of the work.
2. **The code problem.** Even with a matching environment, using more than one machine normally means rewriting the program against a cluster API.
3. **The reachability problem.** Machines on different networks cannot open a connection to one another without a VPN, port forwarding, or a relay that carries every byte between them.

PipedPeer addresses all three without asking the user to change the program. The environment is reproduced exactly by shipping a Nix closure; the program is distributed without modification by intercepting the parallel primitives it already uses; and machines on different networks are joined by a direct path through their routers, with an introducer that only tells each machine where the others are. No machine is a head node: coordination is done per job by the machine that submits it. The contributions of this work are:

1. **Peer-to-peer closure distribution.** Building on Nix's content-addressed store paths, signed closures are pushed between peers, only the store paths a receiver lacks are sent, and distributed work is offered only to peers that can run the closure (Sections 4.6 and 4.7).
2. **Transparent interception with measured work sharing.** Parallel primitives are substituted at import time, parallel maps are divided by a makespan solver over measured rates, and numerical work is gated by a cost model (Sections 4.9 and 4.10).
3. **Joining across networks without a relay**, through an introducer that only exchanges addresses and that every daemon serves (Section 4.2).
4. **Placement without a scheduler**: filters and a bounded score evaluated in the submitting client, with a three-phase lease (Sections 4.4 and 4.5).
5. **A fallback that preserves the result on every distributed path** (Section 4.11).
6. **Speed-weighted data-parallel training through the daemon**, with exact or fast synchronisation chosen by ring size (Section 4.9.2).

The two words of the title are used in a precise sense. Coordination is *decentralised*: every daemon serves the introducer for its own cluster, placement is computed in the submitting client, and a training ring's lead is simply the node that runs rank 0 of that job, so no machine holds a role that the others depend on. It is *recursive* in that division repeats at each level, machines and then cores (Section 4.9).

1.2 Objectives

1. To run an unmodified Python script on the best available machine, with results appearing in the user's working directory as though the script had run locally.
2. To reproduce the script's environment exactly on that machine, sending only the store paths that machine does not already hold.

3. To choose the machine from live capacity, treating architecture and GPU availability as hard requirements and load, memory and hardware as a ranking.
4. To distribute the parallel work inside the script with no code changes, only when doing so is expected to be faster than running it locally, and in proportion to each machine's measured throughput.
5. To join machines on different networks into one cluster with a single command, with no VPN, no port forwarding and no relay in the data path.
6. To authenticate every request, encrypt traffic between machines, isolate each job, and survive the loss of any worker without losing the run.
7. To require no manual installation on a new machine: a single binary, whose setup command installs what else is needed, without root if necessary.

1.3 Scope

Within scope and implemented are a Go daemon and CLI in a single binary; discovery on the local subnet and an address book of every path to every node; joining across networks by an introducer in every daemon, router port mapping, hole punching and QUIC links; a shared secret, pinned TLS and signed closures; reproducible Nix environments transferred as a path-level difference; placement by hard filters and a score, with a three-phase lease; per-job `crun` sandboxes under memory ceilings, with GPU passthrough and without root; transparent interception of `multiprocessing.Pool`, pandas, NumPy, `joblib` and scikit-learn, and torch data-parallel training; and Android phones as workers for parallel chunks. Out of scope are per-user authorisation (a cluster shares a single secret; Section 4.3), runtimes other than Python, relaying traffic for a peer that no direct path can reach (such a peer is reported and routed around), and any incentive mechanism for contributing capacity.

1.4 Technologies Used

PipedPeer is written in Go, so that the client, the daemon and the coordinator ship as one static binary with no runtime to install, and it relies on established components for everything that is not specific to the problem: Nix for reproducible environments, `crun` and cgroups for isolation, QUIC and TLS for transport and security, and SQLite for the address book. The interception shim is written in Python, the language it intercepts. Table 1.1 lists each technology and its purpose.

Table 1.1: Technologies used in PipedPeer

Technology	**Category**	**Purpose**
Go 1.25	Core language	Daemon, CLI and coordinator in one static binary
`go-chi/chi`	HTTP	Routing for the daemon API
`gorilla/websocket`	Streaming	Live output from a running job
`quic-go`	Transport	Encrypted, multiplexed peer links over the punched UDP socket
`crypto/tls`, Ed25519	Security	Pinned TLS on the daemon port; node identity and signing keys
Nix (pinned)	Environments	Content-addressed closures; the store path is the cache key
`crun`	Sandboxing	OCI container per job, including GPU device passthrough
cgroup v2, `systemd-run`	Resources	Memory ceilings for the daemon and for each job
zstd (`klauspost/compress`)	Compression	Workspace, results and request bodies
`modernc.org/sqlite`	Storage	Address book, in pure Go so the binary stays static
`gopsutil`	Metrics	CPU, memory and process-tree measurements
Cobra; Bubble Tea, Lip Gloss	CLI, TUI	Command structure; live dashboard
`zerolog`	Logging	Structured, levelled logging
Python, cloudpickle	Interception	The shim, embedded in the binary; shipping closures by value

1.5 Assumptions and Constraints

- Every participating machine runs the same binary. `pipedpeer setup` installs Nix and `crun` when they are missing, and can do so without root by using a private Nix store.
- Workers run Linux, since `crun`, cgroups and the Nix store are Linux-specific; phones run Linux user space under Termux.
- Machines in a cluster share a single secret, created by the first machine's setup and carried to the others by an invite code.
- Joining across networks needs one reachable member or public introducer. Introducers carry no job data.
- Two machines between which no direct path exists cannot work together.
- The submitting process holds the lease of a parallel script and must remain alive for its duration; a single remote job can be detached.

1.6 Layout of the Report

- Chapter 2 surveys the work PipedPeer draws on.
- Chapter 3 gives the design: architecture, data flow, use cases and the sequence of a run.
- Chapter 4 describes the implementation of each mechanism, from discovery and joining across networks to interception, training and fault tolerance.
- Chapter 5 sets out the evaluation and its results.
- Chapter 6 concludes, with the limitations.
- Chapter 7 lists future scope, and Chapter 8 summarises the research paper drawn from this work.
- Appendix A holds the implementation's size and further measurements.

Chapter 2

Literature Survey

This chapter surveys work in the areas PipedPeer draws on: frameworks for distributed Python, reproducible environments and container runtimes, data-parallel training and gradient compression, lightweight sandboxing at the edge, fault tolerance and failure-transparent programming, portability in the Python ecosystem, scheduling, and collaborative task offloading. Each review states its bearing on the design of Chapters 3 and 4.

"Spark: Cluster Computing with Working Sets" [1] by M. Zaharia *et al.* keeps a working set in memory across parallel operations as resilient distributed datasets, which a lost partition rebuilds from its lineage, and runs iterative jobs up to ten times faster than MapReduce [4]. A program is written against the dataset API and runs on a cluster with a master, which is exactly the setup PipedPeer removes; its hash partitioning by key is the technique PipedPeer's dataframe shuffle uses (Section 4.9).

"Ray: A Distributed Framework for Emerging AI Applications" [2] by P. Moritz *et al.* unifies tasks and actors over a distributed scheduler and a sharded control store, and reaches millions of tasks per second. Ray offers a drop-in `multiprocessing.Pool` and a `joblib` backend, the closest existing answer to PipedPeer's code problem, but each needs a changed import and a running cluster with a head node on one network. PipedPeer substitutes the same primitives without the import and decides per call whether distributing them pays.

"Dask: Parallel Computation with Blocked Algorithms and Task Scheduling" [3] by M. Rocklin expresses NumPy- and pandas-like operations as graphs of blocked tasks run by a dynamic scheduler, so that arrays and dataframes larger than memory can be processed in parallel. Its interfaces mirror the libraries they replace, yet the program must still be written against Dask's collections. The out-of-core `read_csv` and row-block matrix products of Section 4.9 follow Dask's blocking idea while leaving the original pandas and NumPy calls in place.

"Towards Scalable Dataframe Systems" [5] by D. Petersohn *et al.* formalises the dataframe and its algebra, and presents Modin, which keeps the pandas API while partitioning the frame and running its operations in parallel on Ray or Dask. Modin is the nearest precedent for distributing pandas without rewriting it, yet it still needs

a changed import and a running engine. BOINC [6] showed that volunteers' idle machines can serve real computation, but as a platform for specific projects with their own servers rather than a general executor.

"Nix: A Safe and Policy-Free System for Software Deployment" [7] by E. Dolstra *et al.* stores every package under a path named by a cryptographic hash of all its build inputs, so that versions coexist, dependencies are complete by construction, and a package's closure can be copied between machines exactly. With the package set pinned, that path names an environment identically on every machine, which makes it a cluster-wide cache key and lets a transfer send only the paths a peer lacks (Section 4.7). Nix itself decides nothing about where a job should run or how it reaches that machine.

The OCI Runtime Specification [8] defines a container as a root filesystem plus a configuration of namespaces, mounts, cgroups and devices, and `crun` [9] implements it in C with a small footprint and a fast start. Used directly, without an image format or a container daemon, the specification gives each job a minimal read-only root over the Nix store and a device model through which a GPU can be passed in (Section 4.8).

Five studies examine WebAssembly as a lighter sandbox at the edge. Sekigawa *et al.* [10] run WebAssembly modules as isolated compute units without a container daemon; Hoque and Harras [11] name an immature system interface, limited file-system and networking APIs and absent GPU support as the blocking gaps; Kakati and Brorsson [12] report 70–90% of native speed across x86_64, ARM64 and RISC-V; and Bachmeier *et al.* [13] and Khelifa *et al.* [14] find GPU acceleration immature and memory the dominant constraint for AI inference. The argument that a lightweight sandbox can replace a full container runtime carries over, but CPython with native extensions and CUDA libraries cannot run under WebAssembly, so a native environment is shipped as a Nix closure and run under an OCI runtime instead, at 15 to 21 ms more per job than `bwrap` (Section 5.4).

"PyTorch Distributed: Experiences on Accelerating Data Parallel Training" [15] by S. Li *et al.* describes DDP, which replicates the model, buckets gradients and overlaps their all-reduce with the backward pass, scaling near-linearly to 256 GPUs. It assumes ranks that open connections to one another on their own ports, usually inside one data centre. Machines behind different routers cannot, so PipedPeer leaves the model and optimiser to the script and routes the gradient exchange through its daemons (Section 4.9.2).

"8-Bit Approximations for Parallelism in Deep Learning" [16] by T. Dettmers compresses gradients to 8 bits for transfer while keeping model accuracy, and "1-Bit

Stochastic Gradient Descent" [17] by F. Seide *et al.* quantises them to a single bit, carrying each step's quantisation error into the next (error feedback) so that convergence stays close to full precision. The two combine in PipedPeer's wire format: 8-bit values with one scale per block of 4096 and error feedback in both directions, about a quarter of the float32 volume.

"Don't Use Large Mini-Batches, Use Local SGD" [18] by T. Lin *et al.* shows that workers taking several local steps between averages communicate less and, in post-local SGD, generalise better than large-batch training. The observation explains why a periodically averaged ring can finish slightly below one device's loss (Section 5.6).

"DiLoCo: Distributed Low-Communication Training of Language Models" [19] by A. Douillard *et al.*, the work closest to PipedPeer's training path, trains replicas independently for hundreds of steps and merges their parameter changes with an outer Nesterov step, matching synchronous training on a language model while communicating about 500 times less. Fast synchronisation takes the same structure for links between homes, but at its intervals of 2 to 64 steps the outer momentum multiplies the script's learning rate, so the merge is a plain average (Section 4.9.2).

"PowerSGD: Practical Low-Rank Gradient Compression for Distributed Optimization" [20] by T. Vogels *et al.* approximates each gradient matrix by a low-rank product found with one step of power iteration, with error feedback, and matches full-precision accuracy at high compression. Of ten methods tried at 4, 8 and 11 ranks, rank-16 PowerSGD on exact synchronisation alone kept every test model within 1%, yet on a small transformer's narrow matrices it sends more bytes than 8-bit values (Section 5.6), which is why it is not the default.

Li and Fujita build an IoT edge server on Elixir and OTP, where supervisor trees restart failed processes [21], and combine Elixir with an event-driven architecture and MQTT publish-subscribe messaging for high-frequency telemetry [22]. A failed job in PipedPeer is likewise placed again on the next-best node with back-off, but no supervisor process exists: recovery follows from lease expiry (Section 4.5). Closures, workspaces and results are bulk transfers that need streaming and back-pressure, so dispatch uses plain HTTP with a WebSocket for output rather than a message broker.

"Programming Models for Failure-Transparent Distributed Systems" [23] by J. Spenger defines failure transparency — masking failures fully from the programmer — independently of a particular semantics, proves it for a model of stateful dataflow streaming, and builds two programming models on that base. Failure transparency is PipedPeer's central invariant: the user's script contains no failure handling, so every remote path must degrade to a working local one, enforced here

by layered fallback and tests rather than by proof (Sections 4.11 and 5.7).

"Python Array API Standard" [24] by A. Meurer *et al.* standardises an array interface so that the same code runs on NumPy, CuPy or JAX without modification. Its goal, portability without code change, is PipedPeer's, but the mechanism is the mirror image: the standard asks libraries to converge on one interface that users then adopt, while interception changes nothing the user writes and substitutes the implementations behind the interfaces the script already uses.

"iDDS: Intelligent Distributed Dispatch and Scheduling" [25] by W. Guan *et al.* assigns workflow tasks to heterogeneous resources from real-time CPU, memory, data-locality and dependency constraints, driven by a task graph. Its scoring-based assignment is comparable to PipedPeer's placement function (Section 4.4), with two differences: PipedPeer schedules whole scripts rather than task graphs, and its scoring runs in the submitting client, with no scheduler process. Data locality, which iDDS treats as first-class, is left to future work.

"Task Offloading in Multi-Hop Relay-Aided Multi-Access Edge Computing" [26] by Y. Deng *et al.* formulates joint task assignment and relay selection to minimise completion time under transmission and energy constraints. Its comparison of transmission time against local computation time has the same shape as PipedPeer's cost model (Section 4.10), which measures link bandwidth at run time rather than assuming a channel model, and for parallel maps fits each peer's setup time and throughput from the work it has returned.

"Multi-Agent Deep Reinforcement Learning Based Dynamic Task Offloading" [27] by H. He *et al.* lets each device learn an offloading policy from local observations without a central coordinator, outperforming centralised heuristics on the energy-delay trade-off. A learned policy needs training data and a training phase, which is at odds with a tool that a new machine joins by running a single binary; a deterministic rule-based model is used instead, and a learned policy is a natural direction once a deployment has history.

"Joint Optimization of Multi-User Partial Offloading and Resource Allocation" [28] by D. Yong *et al.* optimises the split between locally executed and offloaded portions of a task and finds partial offloading consistently better than all-or-nothing. Interception offloads partially in this sense: it splits a script's work at the granularity of pool chunks, shuffle buckets and matrix row blocks while the rest runs locally, choosing the split by minimising completion time over measured rates (Section 4.9).

"Energy-Efficient Collaborative Task Offloading in MEC" [29] by S. Wang *et al.* has devices cooperate by relaying tasks towards edge servers, with a learned policy that

cuts total energy. Its symmetric model, in which every device is both client and server, matches a single binary in which any node may submit or accept work, but PipedPeer assumes no edge servers and forwards a map once, by a fixed rule rather than a learned policy.

"Blockchain Enabled Task Offloading" [30] by C. Li *et al.* enforces service agreements between task submitters and edge workers with smart contracts, preventing free-riding among parties that do not trust one another. PipedPeer instead trusts every holder of the cluster secret (Section 4.3), so it needs no contract layer but cannot police an admitted participant.

"Congestion-Aware Distributed Task Offloading Using Graph Neural Networks" [31] by Z. Zhao *et al.* trains a graph neural network to produce distributed offloading policies from local neighbourhood observations. PipedPeer scores each node on scalar metrics and is blind to topology when it places a whole job; link cost is measured only where work is divided. Feeding a graph-structured view into placement is the most concrete extension this survey suggests.

The survey leaves a gap that PipedPeer fills. The systems for distributed Python — Spark, Ray, Dask — assume a configured cluster on one network; the offloading literature models the decision of what to move but not the environment, the program or the network path; and the sandboxing work stops at a single node. None runs an unmodified script across machines behind different routers.

Standards for cross-network operation. Joining machines on different networks (Section 4.2) rests on established standards. Address discovery uses the reflexive-address idea of STUN [32], and a router's mapping and filtering behaviour is classified in the terms of RFC 5780 [33]. Port mapping follows NAT-PMP [34], its successor PCP [35], and UPnP IGD [36]. Simultaneous UDP probing from the socket that will carry the connection is hole punching, as analysed by Ford, Srisuresh and Kegel [37], whose observation that a router's mapping belongs to a socket shapes PipedPeer's shared-socket design. Links are QUIC [38] secured by TLS 1.3 [39], node identities are Ed25519 keys [40], and archives are compressed with Zstandard [41].

Table 2.1 summarises the surveyed work.

Table 2.1: Summary of the surveyed work

Author	**Year**	**Method**	**Gap for PipedPeer**
Anderson [6]	2004	Volunteer computing on idle machines	Platform for specific projects, not a general executor
Dolstra et al. [7]	2004	Store paths named by a hash of their build inputs	No placement or transfer between machines

Author	Year	Method	Gap for PipedPeer
Zaharia et al. [1]	2010	In-memory resilient datasets rebuilt from lineage	Dataset API; cluster with a master
Seide et al. [17]	2014	1-bit quantisation with error feedback	Synchronisation still every step
Rocklin [3]	2015	Blocked task graphs for array and dataframe work	Program written against Dask's collections
Dettmers [16]	2016	8-bit gradient approximation	Synchronisation still every step
Moritz et al. [2]	2018	Tasks and actors over a distributed scheduler	Changed import; cluster with a head node on one network
Vogels et al. [20]	2019	Low-rank gradient compression with error feedback	More bytes than 8-bit on small matrices
Petersohn et al. [5]	2020	pandas API over a partitioned, parallel dataframe	Changed import; running Ray or Dask engine
OCI [8], `crun` [9]	2020	Container runtime specification and a light runtime	Isolation only; no environment or placement
Li et al. [15]	2020	Bucketed all-reduce overlapped with the backward pass	Ranks must reach one another directly
Lin et al. [18]	2020	Local steps between averages	Ranks within one cluster
Hoque and Harras [11]	2022	WebAssembly against containers and VMs	Immature system interface; no GPU
Li and Fujita [21]	2022	Elixir/OTP edge server with supervisor trees	Needs a supervisor process
Douillard et al. [19]	2023	Local training with an outer Nesterov merge	Intervals of hundreds of steps; outer step fails at short ones
Sekigawa et al. [10]	2023	WebAssembly containers at the extreme edge	Cannot host CPython with native extensions
Meurer et al. [24]	2023	A standard array API across libraries	Libraries and users must adopt the interface
Deng et al. [26]	2023	Joint assignment and relay selection in MEC	Assumes a channel model
Yong et al. [28]	2023	Optimised partial offloading	Environment, program and network path not modelled
Li et al. [30]	2023	Smart-contract service agreements	Blockchain infrastructure required
Kakati and Brorsson [12]	2024	WebAssembly across three architectures	10–30% below native speed
Li and Fujita [22]	2024	Elixir, event-driven design and MQTT	Broker messaging, not bulk transfer
Khelifa et al. [14]	2024	Four WebAssembly runtimes for AI inference	Inference only; memory the main limit
He et al. [27]	2024	Multi-agent reinforcement learning for offloading	Needs training data and a training phase
Zhao et al. [31]	2024	Graph-neural-network offloading policies	A learned model that must first be trained
Bachmeier et al. [13]	2025	wasi-nn for edge inference	Immature GPU acceleration
Spenger [23]	2025	Failure-transparent programming models	Proved for dataflow streaming, not scripts
Guan et al. [25]	2025	Scored dispatch of workflow task graphs	Central scheduler; task graphs, not scripts
Wang et al. [29]	2025	Cooperative relaying of tasks towards edge servers	Learned policy; edge servers assumed

Chapter 3

Design

3.1 System Architecture

PipedPeer is built around a long-lived **daemon** on every machine, which handles discovery, admission, execution and result delivery; the `pipedpeer` command is a short-lived client of it, and no state lives in the command.

The property that distinguishes the system from the frameworks of Chapter 2 is that **the coordinator is a library linked into the client, not a service** (Figure 3.1). There is no scheduler process, no head node and no master. A machine is submitter and worker at once, and the same binary does both; each job is coordinated by the machine that submits it, and a training ring's synchronisation by the daemon of its lead rank, chosen per job. A new participant runs `pipedpeer connect` with the one-line code another member's `pipedpeer invite` printed; on one network the machines also find each other unaided. Across networks they meet through an **introducer**, an address book that tells machines where to find each other and carries no job data. Every daemon is an introducer for its own cluster. Two optional roles exist: `chunks-only`, for a phone that takes parallel chunks but not whole jobs, and `introducer-only`, for a hosted server that runs nothing.

The principal subsystems are listed in Table 3.1. All are packages under `src/internal/`; the remainder are supporting code (process management, setup, paths, logging).

3.2 Data Flow Diagram

Level 0 — Context Diagram

At the highest level (Figure 3.2) the system has three external entities: the **user**, who submits a script and receives output and result files; **peer nodes**, which supply capacity and return computed results; and an optional **introducer**, which exchanges candidate addresses so that peers on other networks can reach one another.

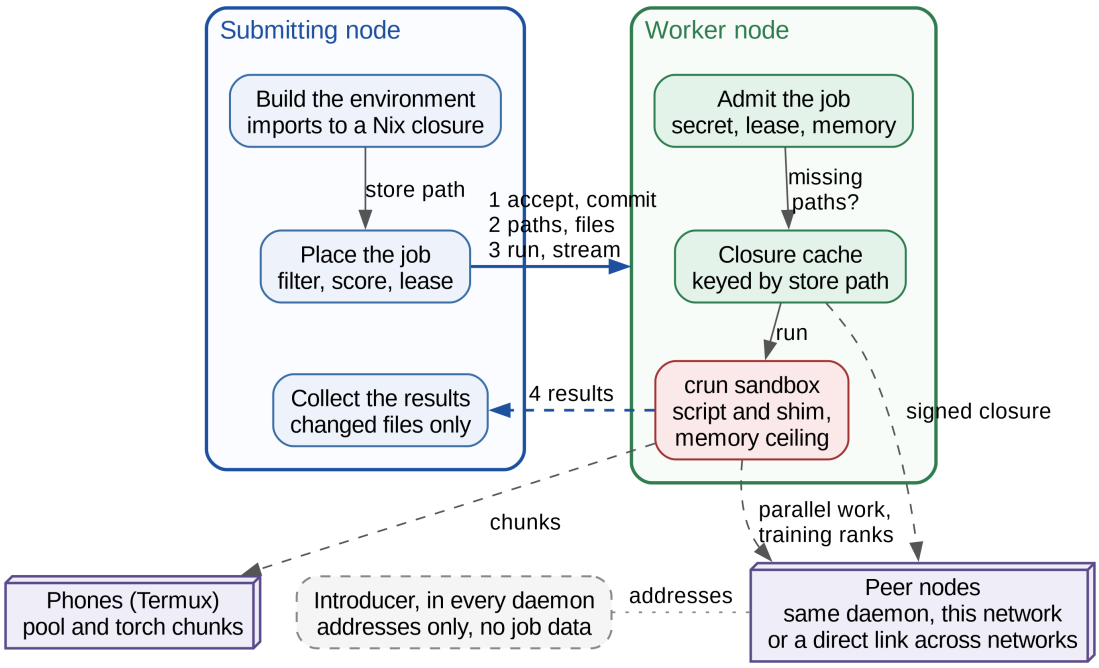


Figure 3.1: PipedPeer system design. The submitting side builds the environment and places the job; the worker admits it, caches the closure and runs it sandboxed under a memory ceiling. Both roles are the same binary. Peers on other networks are reached over direct QUIC links that an introducer helps establish but never carries; every daemon is an introducer for its own cluster. Phones take parallel chunks.

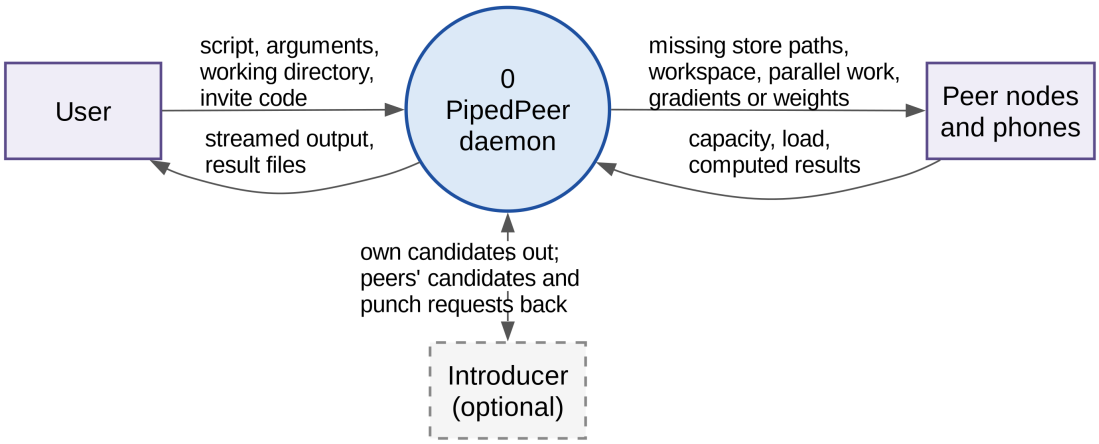


Figure 3.2: Level 0 DFD — context diagram. The introducer is dashed because the system works without it on one network.

Table 3.1: Subsystems and their responsibilities

Package	**Responsibility**
`coordinator`	Filters, scores and selects a node; takes the lease; retries on failure
`daemonapi`	HTTP and WebSocket surface, lease manager, sandbox, pool executor, gradient reduction, closure caches, node poller and address gossip
`schedule`	Makespan-optimal division of a chunk across measured devices
`ddpplace`	Choosing the ranks of a training ring by measured throughput
`discovery`	UDP multicast and broadcast probes, TCP health sweep fallback
`internet`	Peer manager across networks: register with every introducer, punch, link, migrate, forward; the in-daemon introducer
`direct`	Shared UDP socket, candidate race, birthday-collision fallback, QUIC links
`rendezvous`	The introducer's address book and punch-request forwarding
`portmap`	Router port mapping by PCP, NAT-PMP and UPnP-IGD
`authtoken`, `tlsid`	Shared-secret middleware; certificate from the node key, pins by node id, TLS and HTTP on a single port
`clustercfg`, `invite`	The one configuration file (secret, introducers, role); the invite code
`identity`, `nixsign`	Ed25519 node key, whose fingerprint is the node id; signing closures and the trusted-key list
`narpack`, `nixstore`	Signed closure archives and cache pruning; system or private (rootless) Nix store
`cgroups`	Memory ceilings for the daemon and for each job
`nodestore`	SQLite address book: every path to every node, last observed capacity
`nixgen`, `pythondeps`	Flake generation, wheel pins and the interception shim; import scanning and resolution
`resourceest`, `runrec`	Tiered memory estimate; the run record of timed steps
`heartbeat`, `gpu`	Capability and load reporting; vendor GPU detection
`service`, `warmup`, `doctor`, `phone`	Autostart unit; background environment builds; diagnostics; phone worker

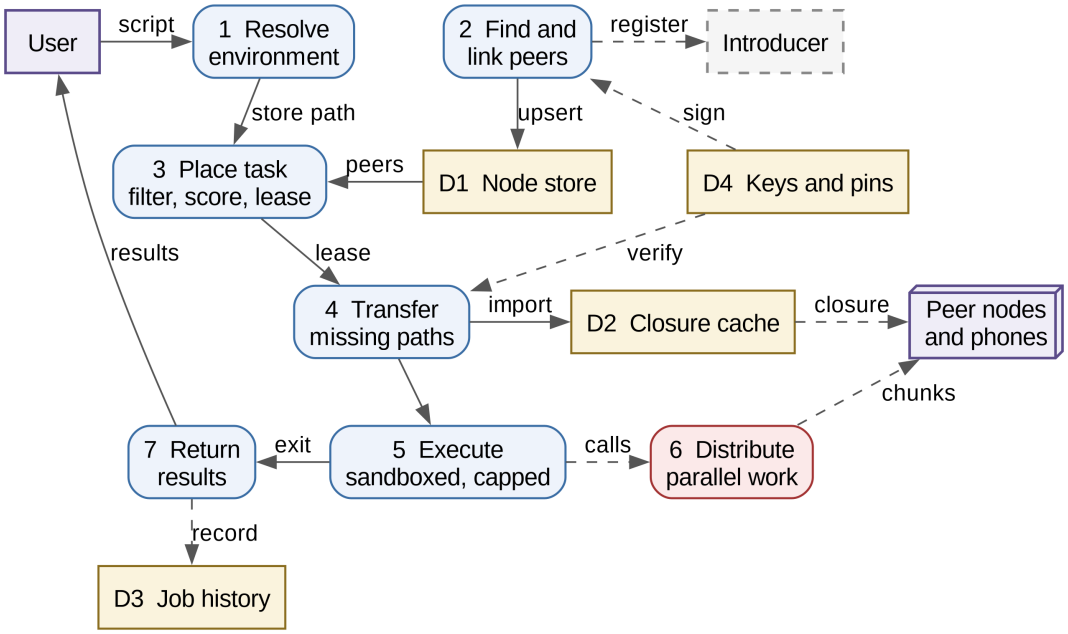


Figure 3.3: Level 1 DFD — seven processes over four data stores.

Level 1 — System Decomposition

Figure 3.3 decomposes the system into seven processes:

- **1 Resolve environment.** Scan the script's imports, generate a flake, build the closure.
- **2 Find and link peers.** Probe the subnet; across networks, publish candidates to the introducer, punch, and register each direct link as a node.
- **3 Place task.** Apply hard filters, score the survivors, reserve a lease on the best.
- **4 Transfer.** Upload the workspace, and of the closure only the store paths the worker lacks; signed between daemons.
- **5 Execute.** Run inside a `crun` sandbox under a memory ceiling, streaming output back as it is written.
- **6 Distribute parallel work.** Share chunks among peers by measured throughput, subject to the cost model.
- **7 Return results.** Send back changed files and a deletion manifest.

The four stores are the address book, the closure cache, job history, and the identity and trust store (node key, pinned certificates, trusted Nix keys).

3.3 Use Case Diagram

Figure 3.4 shows who uses the system and for what; machines are actors as well as people.

- **User:** runs a script on the cluster, asks for a GPU or a memory floor, trains with data parallelism, and inspects jobs and nodes.
- **Node operator:** sets up a machine, invites another machine, and connects a machine with an invite code.
- **Peer daemon:** accepts and leases work, serves signed closures, executes parallel chunks, and introduces its cluster's peers to one another.
- **Phone (Termux):** joins under `proot` and Nix and executes parallel chunks.
- **Hosted introducer:** registers nodes and forwards punch requests; it carries no job data.

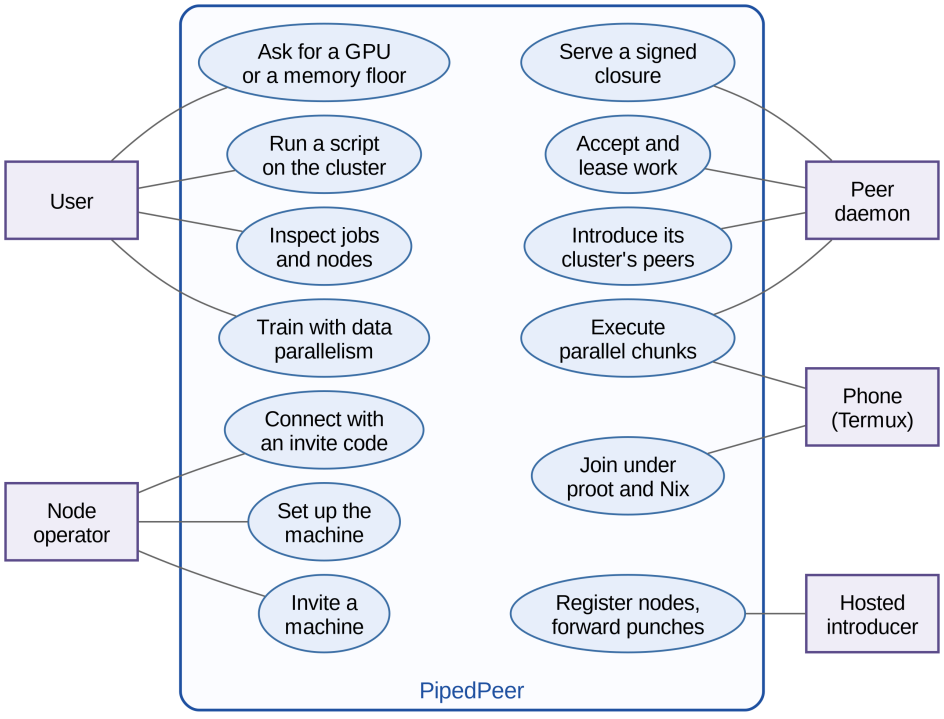


Figure 3.4: Use cases. A peer daemon is an actor in its own right, because every node serves closures, executes chunks and introduces its cluster's peers; a phone executes chunks; a hosted introducer only registers nodes and forwards punch requests.

3.4 Sequence Diagram

Figure 3.5 follows one job from submission to results:

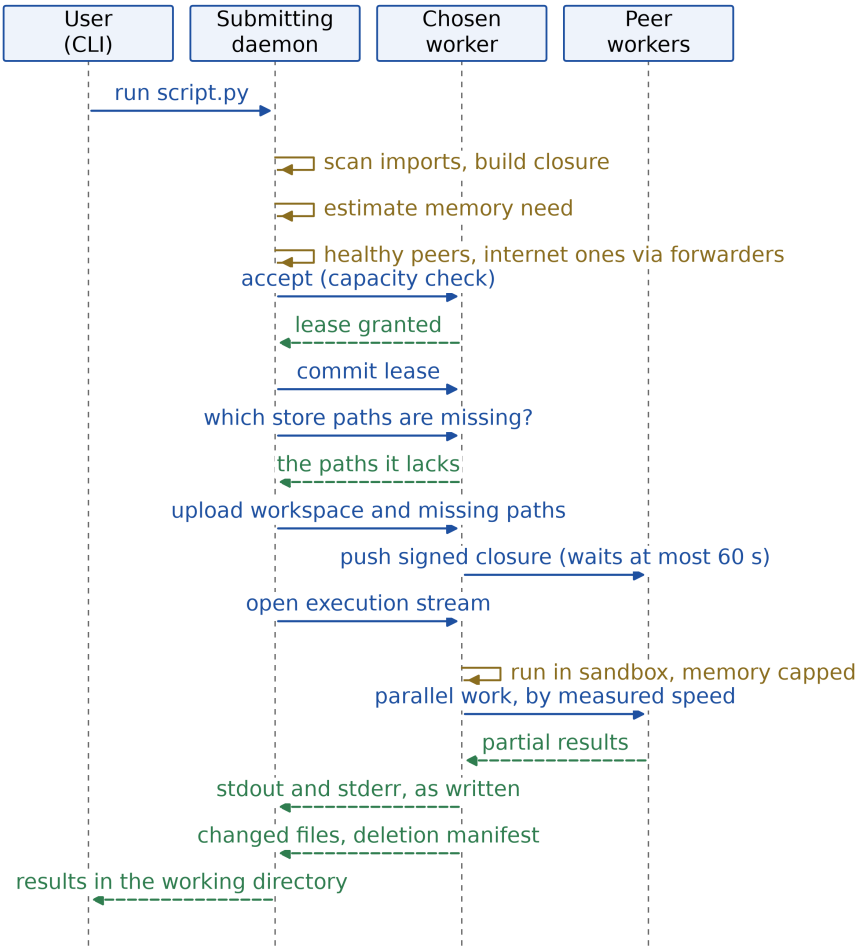


Figure 3.5: End-to-end execution of one job.

1. The user runs `pipedpeer run script.py`.
2. The client scans imports, builds a Nix closure locally, and estimates the job's memory requirement.
3. It asks its own daemon for the healthy peers, including peers on other networks, reached through local forwarders onto direct links.
4. The coordinator filters and scores them and sends `accept` to the best candidate, which checks its capacity and reserves it.
5. The client sends `commit`, which turns the reservation into a running lease.
6. It asks whether the worker can already run the closure; if not, it asks which store paths are missing and sends only those, then uploads the workspace.
7. The worker pushes the closure to peers that lack it, so that they can take a share of the parallel work.
8. Execution proceeds inside a sandbox under a per-job memory ceiling, with output

streamed back as it is written.

9. If the script uses a parallel primitive, the shim distributes it subject to the cost model.
10. On exit, changed files and a deletion manifest return as a compressed archive and are extracted into the user's directory.

Every request in this sequence carries the cluster's shared secret and travels over pinned TLS on a local network or inside a QUIC link across networks.

Chapter 4

Implementation

4.1 Node Discovery and the Address Book

Node discovery merges addresses from three non-blocking sources into a single table.

UDP probes. The daemon opens two sockets on UDP port `4242`, one joined to multicast group `239.255.0.99` and one bound for broadcast, both with address and port reuse. The probe is a fixed magic string and the reply carries a JSON node descriptor; it is a bare request-response probe, not mDNS. The multicast socket exists because a UDP broadcast to a shared port is delivered by the kernel to only one socket, so two daemons on one host would never see each other; multicast with port reuse reaches all of them.

TCP fallback. Access points with client isolation drop broadcast traffic without error. When the UDP probe returns nothing, the daemon sweeps each local interface's prefix (container and VPN interfaces skipped) with concurrent health probes, and stops once any daemon answers. A reply's host is taken from the packet's source address, never from the address the peer advertised at startup.

The address book. The node store keeps every node under its node id, the fingerprint of its key, together with every *path* to it: its kind (LAN, internet, gossip, self), host, port, and when it was last seen, last worked and last failed. A sighting only adds a candidate. A path becomes the active one only after the poller has reached it and the certificate's key matched the node id; an active path that still answers is kept, and a verified LAN path beats a verified internet path. Members also gossip their own addresses, signed with their node keys, over the authenticated API; a gossiped address is a candidate, never a fact.

The poller tries the active path first; a node with no verified path for 15 minutes is removed, and a sighting alone never refreshes it.

4.2 Joining Across Networks

A broadcast cannot leave the local network, and a machine behind another router accepts no inbound connection. Figure 4.1 shows how two such machines come to

share a direct, encrypted link without either opening a port by hand and without any machine relaying their traffic.

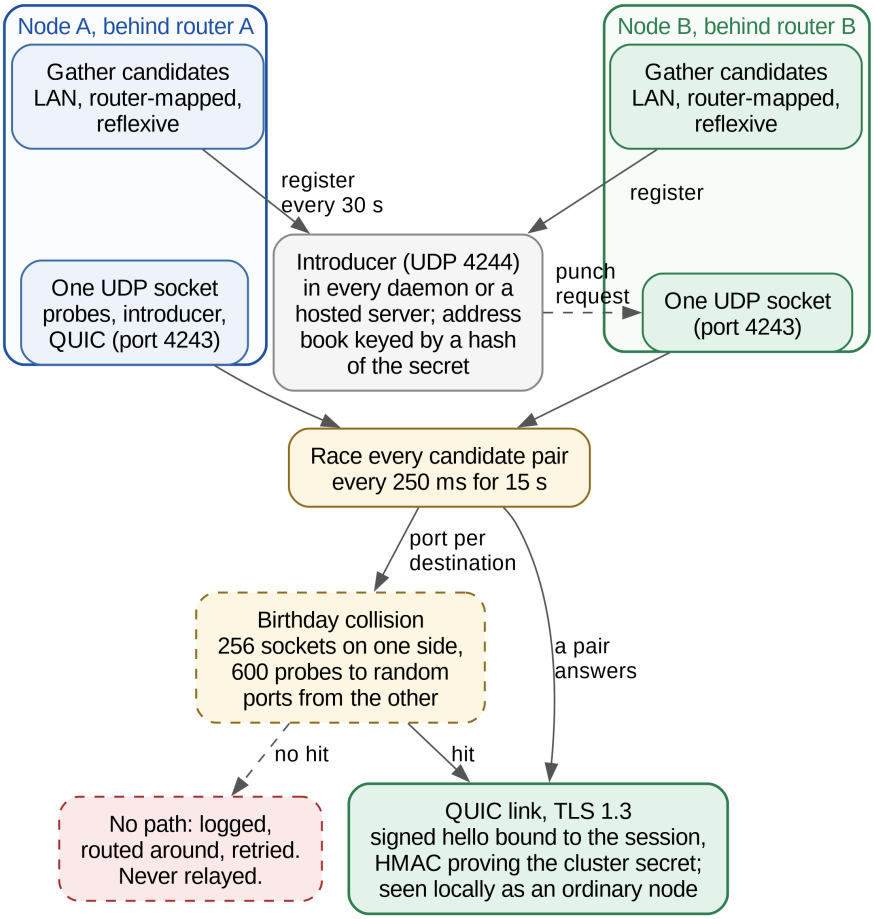


Figure 4.1: Joining across networks. The introducer exchanges candidate addresses and forwards one punch request; the two nodes then race their candidate pairs over a single UDP socket, fall back to a birthday collision when one NAT allocates a port per destination, and carry everything over a mutually authenticated QUIC link exposed locally as an ordinary node.

One command per machine. `pipedpeer invite` prints a single line, `pipedpeer connect <code>`, whose code carries the cluster secret and up to six introducer addresses with a checksum, so a mistyped code is refused as damaged rather than tried. `connect` saves both, restarts the daemon once, waits up to 60 s, and ends with one of three results: connected, naming the machines (exit 0); partly connected, giving each missing machine's reason and fix (exit 2); or failed, with what to do (exit 1).

An introducer in every daemon. Every daemon holding a secret serves the introducer for its own cluster on UDP 4244 and refuses other clusters. A node registers with every introducer it knows — those in its invite code and members it has verified — periodically and at once when its own address changes, so the cluster

survives the loss or readdressing of any one member. A public server can join as an introducer that takes no jobs. Nodes register their candidates under a cluster identifier derived from the secret by hashing, so an introducer never holds the secret itself. A `connect` request makes it forward a single punch datagram to the target, carrying only the candidates the caller itself registered, rate-limited per source. It carries no job data.

Candidates. Each node offers three kinds of address: its LAN addresses; a port its router agreed to forward, requested by PCP [35], NAT-PMP [34] or UPnP-IGD [36] and renewed before it lapses; and the reflexive address the introducer observed, in the manner of STUN [32]. A router mapping is withheld when its external address differs from the reflexive one: behind a second layer of NAT, such as a phone tether, the router that answers is not the one facing the internet.

One socket. Punch probes, introducer messages and QUIC share a single UDP socket on port 4243. Sharing one socket is required for correctness, not convenience: a router's mapping belongs to the socket that created it [37], so the connection must arrive on the socket the probes left from. Datagrams are separated by content — probes by four magic bytes, introducer messages by a leading brace, everything else handed to QUIC.

The race. Both sides probe every pairing of their candidates every 250 ms for 15 s, rather than trying candidates in turn, which would add a timeout for every address that cannot work. When one side's NAT allocates a fresh external port for every destination there is no address to publish, and the **birthday collision** [42] runs: that side opens 256 sockets towards the peer, the other sends 600 probes to random ports among the 64,511 above 1024, and the socket that is hit is kept, since its mapping is the reachable one. A round hits one of the 256 mappings with probability $1 - (1 - 256/64511)^{600} \approx 91\%$.

The link. The winning path carries QUIC [38] with TLS 1.3. Inside it each side sends a hello signed with its Ed25519 node key, and each checks that the key's fingerprint is the node it meant to reach and that the cluster identifier matches. The hello signs the session's exported keying material [43] and proves the cluster secret with an HMAC [44] over the same value, so it is valid for one session and one cluster only and cannot be replayed to a third node. The link is exposed to the rest of the daemon as a TCP forwarder on a loopback port, recorded as that node's internet path, and each local connection becomes one QUIC stream to the peer's daemon. Placement, closure transfer and gradient exchange therefore work over the link unchanged.

Roaming. A hello from a key that already has a link replaces that link at once.

The daemon watches its interfaces; on a change it migrates the QUIC connections it dialled to the new address, redials those it accepted, and registers again with every introducer.

Peers without a direct path. No relay is in the data path, since a relay would carry every byte of every job and become the cluster's bottleneck and its cost. A peer with no direct path is logged with the reason, never offered to the scheduler, and retried after a back-off; `pipedpeer status` lists it with that reason.

4.3 Security and Identity

Security has three layers (Figure 4.2).

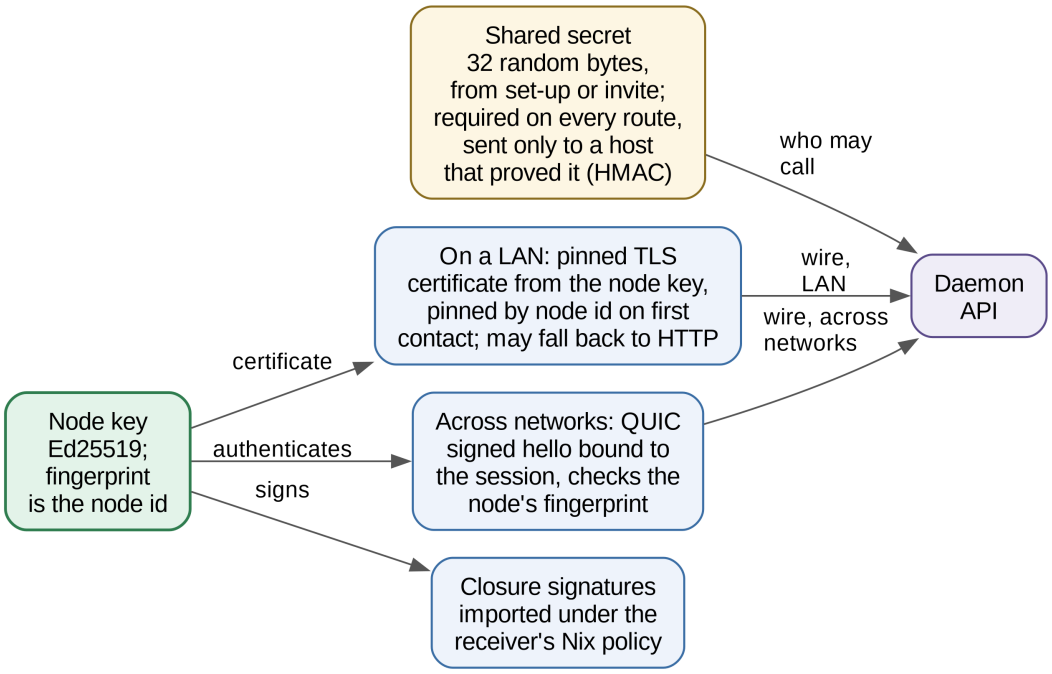


Figure 4.2: The three layers of trust: a shared secret on every API route, pinned TLS (or QUIC across networks) on the wire, and the node key that makes the certificate, authenticates links and signs closures.

The shared secret. The first machine's setup stores 32 random bytes, readable only by the user, before its daemon first starts; `connect` stores the secret from the invite code. Every route on the daemon, `/health` included, requires it, compared in constant time, and outbound requests carry it automatically. Loopback is not exempt, since peers across networks arrive through loopback forwarders. The secret is attached only to a host that has proved it holds it, by returning an HMAC over a challenge. Jobs receive it so that the shim can reach its own daemon, and the web page receives it in the URL fragment opened by `pipedpeer ui`, which a browser

never sends to a server.

Pinned TLS. Each daemon's self-signed certificate is made from its Ed25519 node key. TLS and plain HTTP share the daemon port; the listener inspects the first byte of each connection. Clients try TLS first, and the pin is keyed by node id rather than by address: a certificate whose key matches the expected node id is accepted wherever the node now is, and a different key for a known node id is refused. The scheme is trust on first use and opportunistic: a handshake that fails for a reason other than a pin mismatch makes the client remember that peer as plain HTTP. Traffic across networks is encrypted by QUIC.

Node keys and signed closures. Each node holds one Ed25519 key [40] whose fingerprint is its node id everywhere. The same key authenticates QUIC links, makes the TLS certificate, signs gossiped addresses, and signs closures as a Nix signing key. Nodes publish the public half in their heartbeat, each daemon keeps a list of the keys of the nodes it knows, and `setup`, with consent, adds them to the system Nix configuration as trusted keys; it never disables Nix's signature requirement. A closure sent between daemons in the signed format is imported under the receiver's own signature policy.

A worker learns a new peer's key before Nix does, so the import passes the cluster's keys to that one copy operation, after checking the archive's named signer against the signer's published identity. Because a multi-user Nix daemon ignores keys passed by an untrusted user, setup also installs a root-owned helper that can do nothing but add or remove keys of the cluster's fixed form.

Limits. These protections have significant limits. Every holder of the secret can run code on every node, and jobs can read the secret. TLS on a local network is trust on first use and can be downgraded by an attacker who can break the handshake. The list of keys trusted to sign closures includes nodes found by unauthenticated LAN discovery, and the submitter's own upload is imported without a signature check. PipedPeer therefore suits machines whose owners trust one another, over networks they need not trust.

4.4 Placement and Scoring

Candidates are merged from the address book, live discovery and the local node, deduplicated by node id; a node with a role is never offered a job. Five hard filters run in order: architecture must match the closure; a GPU must be present if one was required; and free memory, free cores and free VRAM must each satisfy the request (Figure 4.3). Every rejection records a reason, shown by

`pipedpeer jobs <id> --explain`.

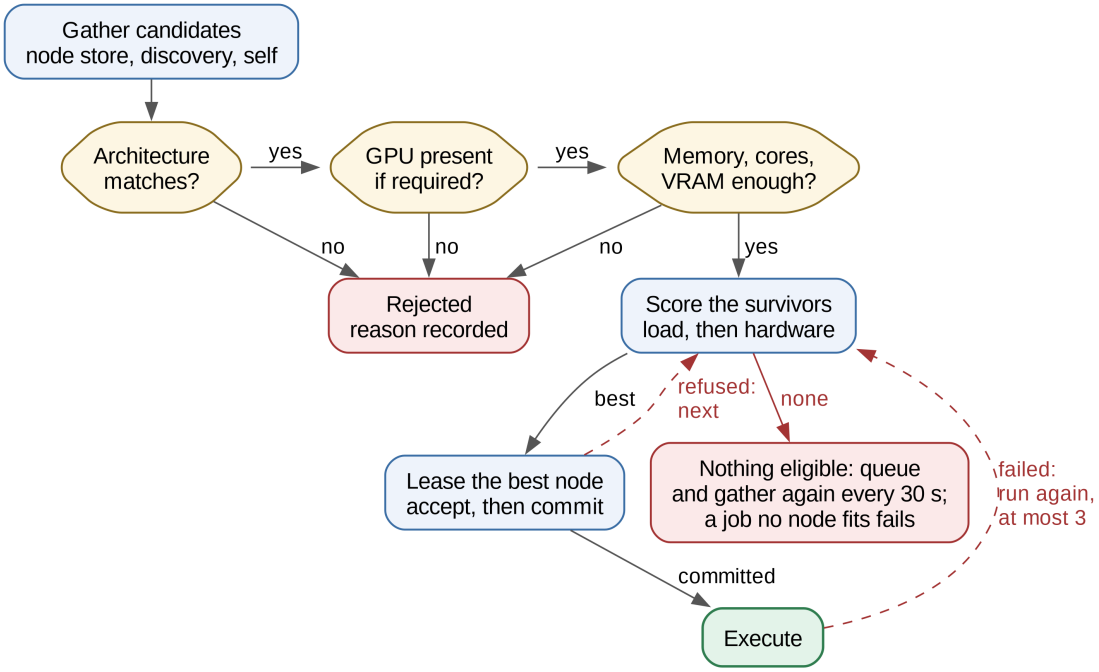


Figure 4.3: Placement: gather, filter, score, lease, retry. A task is never dropped; when nothing is eligible it is queued and retried.

Survivors are scored. Writing u_{cpu} and u_{mem} for CPU and memory utilisation in percent, n_{jobs} for the count of running jobs, $\hat{c}, \hat{f}, \hat{m}, \hat{r}$ for the core count, clock speed, free memory and free-core ratio each normalised onto $[0, 1]$, and $h \in [0, 1]$ for the node's health score, the score is

$$S = \Big(1 - \frac{u_{\text{cpu}} + u_{\text{mem}}}{200} - 0.05\, n_{\text{jobs}} + 0.10(\hat{c} - 0.5) + 0.06(\hat{f} - 0.5) + 0.10(\hat{m} - 0.5) + 0.10(\hat{r} - 0.5)\Big) \cdot h. \tag{4.1}$$

Two properties matter more than the individual coefficients. **Load dominates hardware**: the two utilisation terms can subtract 1.0, while the four hardware terms together move the score by at most ± 0.18, so a fast but busy machine loses to a slower idle one and is left to the person using it. **Missing telemetry is neutral**: normalisation returns the midpoint 0.5 for an absent value, so a node that does not report its clock speed is neither rewarded nor punished. S is clamped at zero.

When a GPU is requested, a second function adds up to 0.8 to S, drawn from total VRAM, device count, free VRAM on the best device and compute capability, less a utilisation penalty. The free memory a node advertises is what it can actually *grant*: its own budget less the reservations of every daemon on the same machine (Section 4.8).

A node that refuses the job's signed closure is left out for the rest of the run, and the client prints the command that fixes it on that node.

4.5 The Lease Protocol

Placement is a three-phase handshake, and its state machine is where crash recovery lives (Figure 4.4).

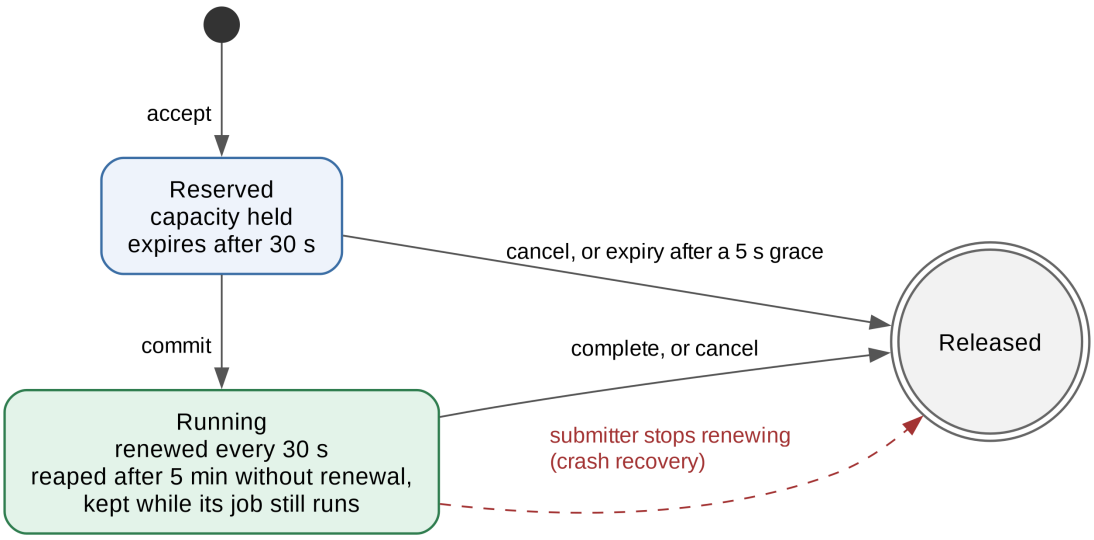


Figure 4.4: Lease states. Recovery is a consequence of expiry rather than a separate mechanism.

An `accept` reserves capacity for 30 s; a `commit` promotes the reservation to running, renewed every 30 s and reaped after 5 minutes without renewal; `complete` releases it, and a sweeper reclaims what has expired. The submitter renews while the environment builds and uploads, so a long build never outlives its lease, and a lease bound to a running job is kept past its expiry.

Two details carry the correctness argument. The concurrency and memory checks and the lease insertion happen **under a single lock**, so two submitters racing for the last slot cannot both succeed. And the concurrency limit counts reserved and running leases together, so a burst of submitters cannot slip through the window between accept and commit.

Recovery then needs no further machinery. A submitter that dies stops renewing, and the sweeper reclaims its slot. A worker that dies stops answering health checks and its in-flight connection fails, which the coordinator treats as a failed attempt and reschedules, with exponential back-off. Waiting for room never drops a task, while a job that *fails* is run at most three times, on untried nodes first. From the

submitter a dead node and a lost network look the same, but the first means run the job elsewhere and the second means wait; the client therefore runs three short probes at once, none needing root — another peer's health endpoint, a registration with the introducer, and a TCP connection to the default gateway — and an answer to any of them confirms that the remote peer, not the local network, failed.

4.6 Reproducing the Environment

Import scanning. The client extracts imports with a line-oriented parser: it strips comments, takes the module from `from X import …`, and splits `import a, b as c` into every module named. It does not see an import performed through `importlib`. Each name is classified as a local file, a standard-library module, or an external dependency, and external names are resolved through a mapping table (`sklearn` to `scikit-learn`, for example) and otherwise looked up as a package of the same name in the pinned package set, with one evaluation for all of them. The scan follows the script's own modules, with a visited set so that import cycles end; a `requirements.txt` beside the script is read, and a `uv.lock` file or an explicit `--pkg` flag override detection. Only what is still unresolved is reported, before anything runs, with the flag that would add it.

Content-addressed environment pinning. Nix [7] names every build output by a hash of its inputs. The generated flake pins an *exact* nixpkgs revision, not a branch. A branch resolves at build time, so two nodes building the same script on different days would produce different store paths and the system would re-ship a multi-hundred-megabyte archive for every job. Pinning makes the store path a stable, content-derived name for an environment, identical across machines and across time, and therefore a cluster-wide cache key (Section 5.3).

Every environment also contains `cloudpickle` [45], so that parallel kernels with no source can be shipped by value. On x86_64 Linux, scikit-learn is pinned to a specific wheel, and torch with CUDA is fetched as a pinned wheel set inside a *fixed-output derivation* with a recorded hash, because building it from source takes hours. The CUDA build is used when a GPU is requested, which is the default when a script imports torch.

Common packages built ahead of time. Environments share Nix store paths, and a transfer sends only the paths a peer lacks. After `connect`, every machine builds in the background one environment importing the usual scientific set (NumPy, pandas, scikit-learn, SciPy, matplotlib, seaborn, tqdm and CUDA torch), so a script seen for the first time shares those paths and builds only what it adds.

4.7 Closure Transfer and Cache Reuse

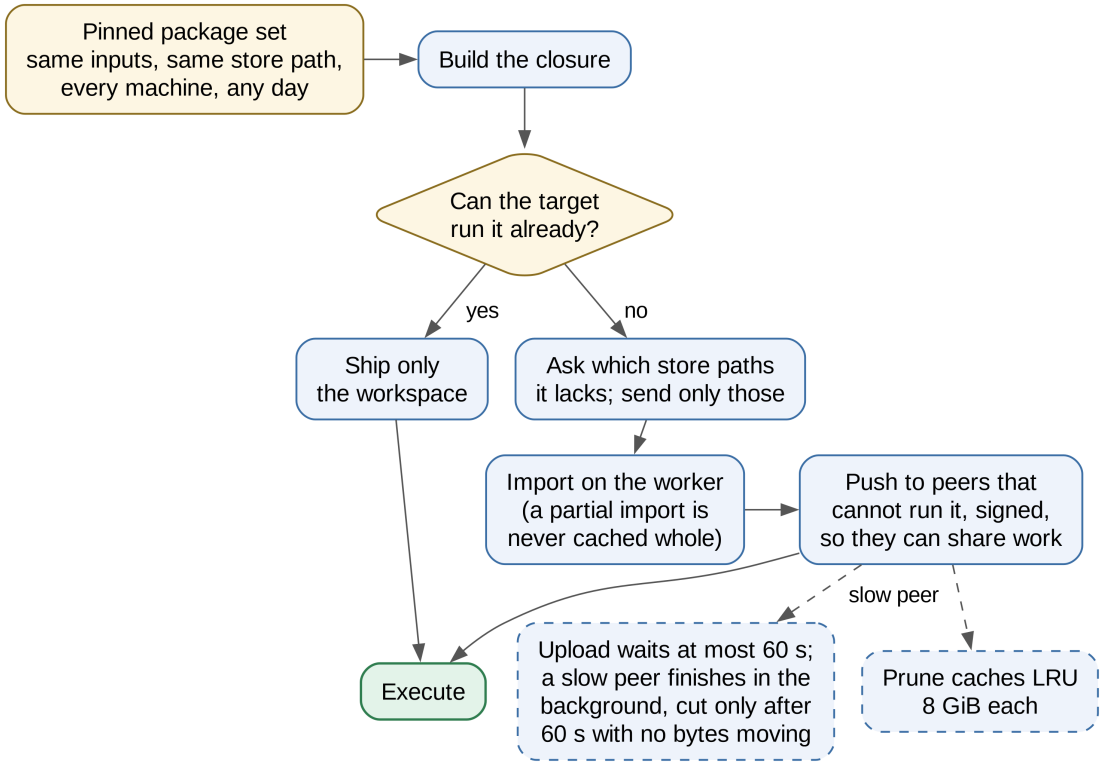


Figure 4.5: The store path is the cluster-wide cache key. Most submissions ship no environment at all, and a related environment ships only the store paths the receiver lacks. The upload never waits more than 60 s for a slow peer.

Before uploading, the client asks the target whether it can already run the closure (Figure 4.5), and only on a negative answer does anything cross the wire. Two questions are needed: whether the archive is in the node's cache, and whether the closure is *materialised*, that is, whether its entry point exists on disk. A deployment with a shared store volume has the closure present without ever having cached an archive, while on a per-node store the entry point appears only after an import; either predicate alone is wrong for one of the layouts.

Path-level difference. A closure is a set of store paths, and two environments that differ by one package share most of them. The client lists the closure's paths and asks the target which it lacks; if some but not all are missing, only those are sent, as a partial import that the receiver never caches as the whole closure. A fresh node may instead fetch public paths from the Nix binary cache; if that download falls well behind the sender's archive, it is abandoned and the sender sends every missing path itself.

Signed transfer between daemons. After an upload, the worker pushes the closure to peers that cannot yet run it. Parallel work is shared only with peers

that can run the closure, so this push is what makes the cluster usable for the distribution of Section 4.9. A peer that advertises the signed format receives one archive containing a signed binary cache — the signature records for every path and the compressed archives for the paths it lacks — produced and imported by Nix's own copy mechanism.

Not waiting for the slowest peer. The upload waits at most 60 s for these pushes. A peer still importing finishes in the background and joins the pool by itself, since each parallel chunk asks afresh which peers can run it; meanwhile it reports "importing", so no second send starts. A background send is cut only when it stops moving. Both closure caches are pruned least-recently-used to 8 GiB each, and workspace and results archives are compressed with Zstandard [41].

4.8 Execution and Isolation

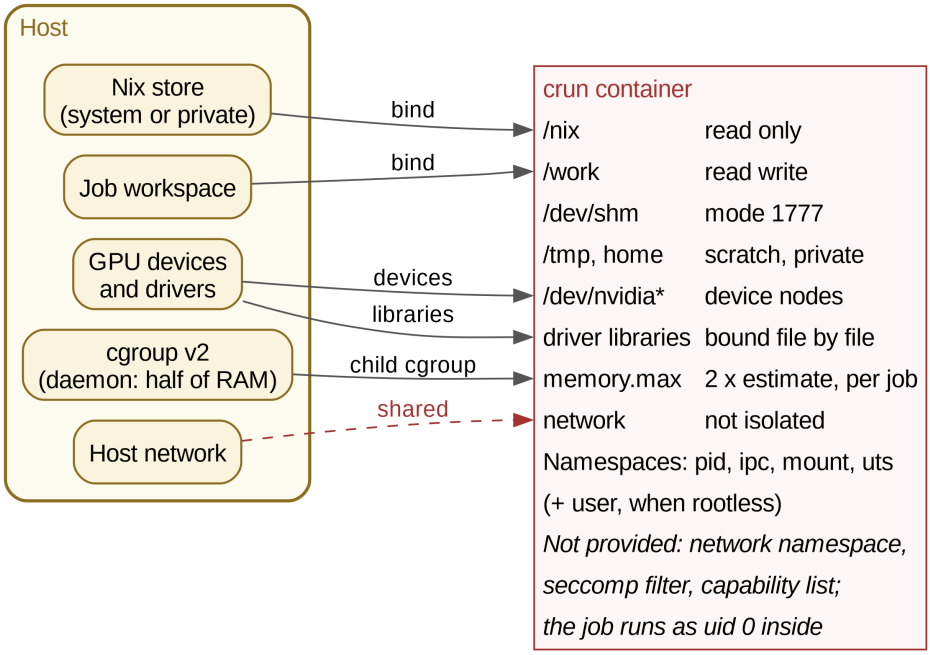


Figure 4.6: What the job can and cannot see. Shared memory is mounted world-writable because Python's `multiprocessing` requires it, and the driver libraries are bound file by file so that the host C library cannot shadow the closure's.

Jobs run under `crun` [9] in an OCI bundle [8] with a minimal read-only root (Figure 4.6): the store read-only, the workspace read-write, a private home, and temporary filesystems for scratch space. Shared memory is mounted world-writable because Python's `multiprocessing` maps POSIX shared memory for its semaphores. GPU support mounts the device nodes and binds the driver libraries *file by file*, because binding the host library directory would place the host C library ahead of

the closure's, and torch would then fall back to the CPU silently.

A memory ceiling for every job. Where the daemon has a usable cgroup v2 hierarchy, each job runs in its own cgroup with a memory limit, and an equal swap limit, of twice its estimated requirement, clamped to what the node can grant but never below what admission approved. When a job is killed and the cgroup's out-of-memory counter has risen, the client says so and how to raise `--mem`.

A memory ceiling for the daemon. Parallel chunks run in warm worker processes in the daemon's own cgroup, so `pipedpeer start` launches the daemon in a user `systemd` scope with a ceiling of half the machine and swap refused. A daemon without a user `systemd`, such as one in a container, runs a watchdog instead that measures the daemon's children and, past the ceiling, kills the largest child and logs it, never the daemon. A hard limit is used rather than a throttling one, because a job held at a throttling limit stalls instead of failing. Daemons on a single machine, recognised by the kernel's boot identifier, subtract one another's reservations, so two of them cannot promise the same memory twice.

Without root. On a machine with no system Nix, setup seeds a private store in the user's data directory and mounts it at `/nix` inside a user and mount namespace, and installs `crun` into the user's own binary directory when no package manager supplies it.

What the sandbox does not do. The job runs as user 0 inside the container and shares the host's network; the specification sets no seccomp filter and no capability list. The namespaces in use are process, IPC and mount, plus hostname where the kernel allows it, and a user namespace when `crun` runs without root. A node without `crun` or user namespaces warns and runs the job unisolated, unless it uses a private Nix store, in which case it refuses.

Results and output. Only files created or modified during the run are returned, with deletions carried in a manifest, so a job cannot overwrite the submitter's own sources; any path that escapes the job's directory is refused on both sides. Output is forwarded as each read arrives, so a progress bar on a remote node animates on the submitter's terminal.

Stopping and detaching. An interrupt or a closed terminal makes the client send one cancel frame; the daemon signals the job's process group, kills it if it is still alive shortly after, and the cancel reaches remote runs, parallel chunks on peers and every training rank. A connection that ends *without* a cancel frame never kills the job, and background mode builds on this: the daemon logs every output frame, a detach closes the channel without a cancel, and `pipedpeer attach` replays the log,

follows live, and downloads the results at the end. A detached job's directory is pruned 24 hours after it ends.

Phones. An Android phone joins under Termux [46] as a worker for parallel chunks, running them from a Nix environment inside a Debian under `proot` [47] where setup can install one, and otherwise with Termux's own Python. Closures go only to peers of the same architecture, so a phone receives the job's package list instead, builds its own environment in the background, and declines the job until it is ready.

4.9 Transparent Interception

Delivery. The shim is a Python package embedded in the binary. It is appended to the workspace archive under a hidden directory, so the user's project on disk is never modified, and the worker points the interpreter's module path at that directory, where a `sitecustomize` module makes CPython import it before the user's first line. Nothing is installed on any node. Nix ships its own module of the same name, which adds the environment's package directories; the shim performs that work first, since only the first module on the path is imported (Figure 4.7). `multiprocessing.Pool` and `ProcessPoolExecutor` are patched at startup, while NumPy, pandas, torch and joblib are patched on their first import through an import hook, so a job that never imports torch never pays for patching it (Section 5.4 measures the startup cost). Table 4.1 lists the intercepted primitives and how each is distributed.

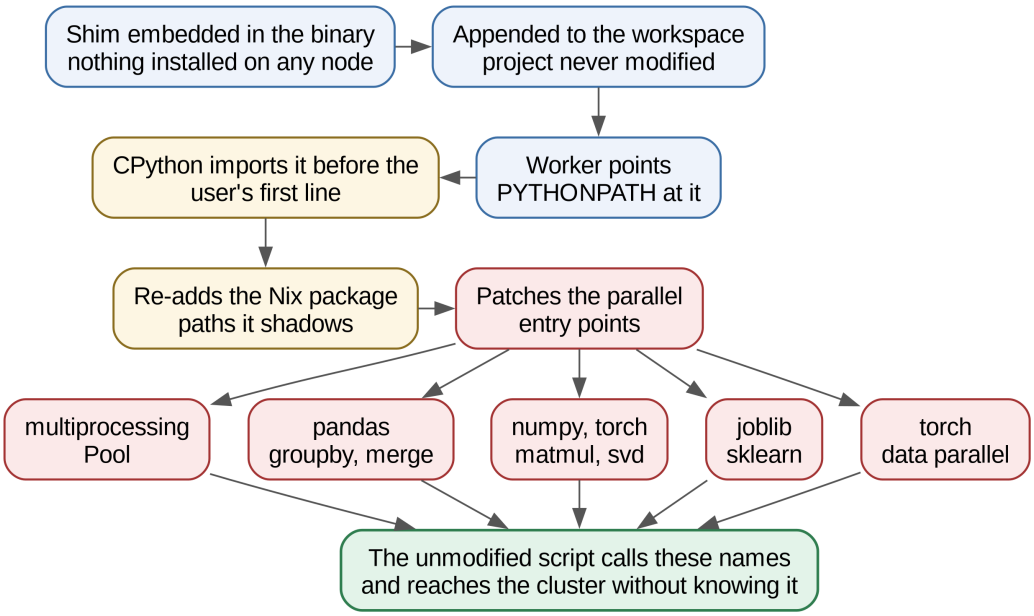


Figure 4.7: How the shim reaches the interpreter and what it replaces.

4.9.1 Parallel Maps

`map`, `starmap`, `imap` and `imap_unordered` go to the cluster (`imap` lazily, in batches), while `apply` and `apply_async` stay local. A small prefix of items runs locally first to measure per-item cost, and a map whose remaining work is under half a second stays local. The kernel reaches the peer in one of two ways. A module-level function, including one defined in the script's `__main__`, travels as its *source*, with the helper functions it calls and a pickle of the module data it reads. A lambda, closure, bound method, partial or decorated function travels *by value* through `cloudpickle`. Anything neither can carry is counted as unshippable and runs locally. Every run can write a **receipt** counting the items that ran remotely and locally, the parts that failed, and the node each part ran on, so a claim about where work ran can be checked.

Sharing work by measured speed. On the daemon, a chunk is divided among peers by a makespan solver. Each device is modelled as a setup time o_i plus a throughput r_i, fitted by least squares from the recent parts it returned; an unmeasured peer is assumed to be as fast as this node, scaled by its core count, since assuming it slow would size its share below anything worth sending. The solver finds the finishing time T at which the devices together complete the work, allocating $(T - o_i)\, r_i$ items to each, and drops any device whose setup time alone exceeds T. On a peer, a part fans out across its cores. Division therefore repeats at two levels, machines and then cores, and a part already split off is never split across machines or forwarded again. A map too large for the node that holds it is the one case of forwarding: it goes whole, once, to a peer that can hold it, and that peer makes the split (Section 4.11).

Slow peers. A part still unfinished past its prediction counts as taking at least that long, so a slow peer's estimate can only grow. Three rules act before the fit has caught up: a split part waiting for its peer starts on another peer with a free slot when that would finish sooner; a part running past its prediction gets a copy on the fastest free peer, the first result winning and the loser cancelled; and a chunk too small to split goes to the peer predicted to finish first. A peer that keeps losing races stops being raced.

Transfer. Each node's chunks in flight follow its measured round trip, and chunk size follows the measured transfer cost. Large functions and globals cross once per peer, by hash; requests are compressed at a level chosen by the measured link speed; and numerical buffers travel outside the serialised request. With `--offload`, every distributed piece goes only to other machines, keeping the sender free for its user.

Interception replaces module functions, so the operator form `A @ B` is not intercepted

Table 4.1: Intercepted primitives and their distribution strategies

Primitive	**Strategy**
`multiprocessing.Pool`, `ProcessPoolExecutor.map`	Measure a few items locally, then race local against remote chunks shared by measured speed
pandas `groupby().agg()`, `merge`, `join`	Hash shuffle on the key; bucket zero local
pandas `read_csv`, `read_parquet`	Out-of-core reads in 64 MiB chunks cut at record boundaries
`numpy.matmul`, `dot`, `tensordot` (2-D)	Split into row blocks, concatenated at the origin
`numpy.linalg.svd`, `eig`	Whole matrix offloaded to one worker
`joblib.Parallel`, scikit-learn `n_jobs`	Batches of delayed tasks, shared arrays sent once per chunk
torch `matmul`, `mm`	Row blocks, gated on size
torch training (`--ddp`)	Gradients or weights averaged by the lead daemon, weighted by samples

for NumPy arrays. A `joblib` call made inside a remote task runs inline on that worker. pandas operations are distributed only above 512 MiB unless forced, because their arithmetic intensity is low.

Why the shuffle is exact. Hash partitioning on the key is the standard technique of MapReduce [4]; PipedPeer uses only that path, never a combiner. Rows are bucketed by a hash of the grouping or join key, so equal keys land on the same node and every key is complete on exactly one node (Figure 4.8). Each node's aggregation is therefore a *complete* aggregation, the origin combines the partials by concatenation, and *any* aggregation specification works, including those with no combiner form. Exactness is of values, not of row order: a concatenation of buckets does not reproduce pandas' join ordering, so a script that depends on the row order of a `merge` result can see a different order.

4.9.2 Data-Parallel Training

Synchronisation over the daemon port. Rather than let torch build its rank-to-rank mesh [15], which needs direct connections on its own ports between every pair of ranks, each rank posts its gradient to the lead node's ordinary daemon port, which works wherever the daemon port works, including over the links of Section 4.2. The daemon *reduces*: gradients arrive as raw frames, are summed in double precision weighted by the number of samples behind each, $\sum_i n_i g_i / \sum_i n_i$, and one averaged gradient is returned (Figure 4.9). An earlier all-to-all design, in

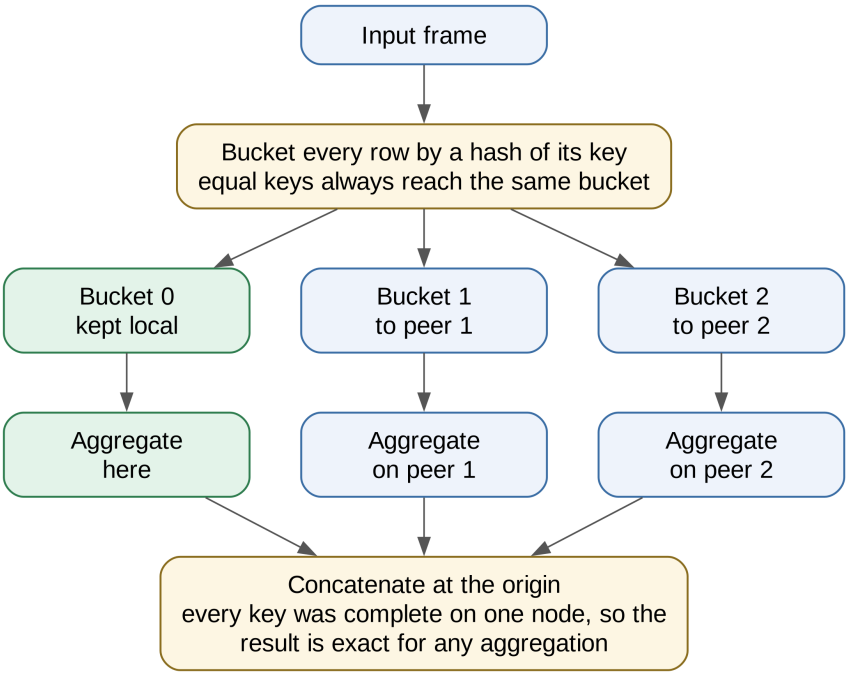


Figure 4.8: Hash shuffle. Correctness follows from key locality, not from combiner algebra.

which every rank received every other rank's gradient, ran a two-machine job in 62.9 s against 2.9 s on one machine. Gradients travel as 8-bit integers with one scale per block of 4096 values, the rounding error carried into the next step in both directions (error feedback) [16, 17]; `PIPEDPEER_DDP_INT8=0` sends 16-bit floats instead. In exact mode a ring of three or more machines, each reachable directly, reduces in a butterfly instead: each rank's daemon averages one shard of the gradient, so each machine's link carries $2(N-1)/N$ of a gradient in each direction (its rank's shards and its daemon's replies), where through the lead the lead's link carries $N-1$ in each direction. Ranks that share a machine instead average there first and send one synchronisation for the machine. A ring of two, and every fast-mode round, goes through the lead.

Choosing the ring. Each candidate's throughput is probed, and the ring admitted is the prefix of the fastest that maximises $k \cdot r_k$ — the ring's worth if every rank ran at the slowest admitted rate — using one device kind only, GPUs when any are wanted and present, with the lead on a GPU node. Each rank's shard and batch slice are sized by its share, and the slices sum to the script's own batch, so the learning rate and the mathematics are the script's. Every rank sends a manifest of its parameters to the opening barrier, and a mismatch stops every rank with one message. The daemon periodically refits the shares from observed step rates, applying them at the next epoch boundary when a share has moved enough to matter, and it can drop

the slowest rank — never rank 0, never below two — when that rank adds more synchronisation time than computation. A round that a straggler has not joined completes without it after a timeout.

One GPU or a split. After a warm-up, the lead daemon compares the measured step time of the split with that of the fastest rank alone, and either keeps the ring or releases every rank but the fastest, which exit with a code meaning "released" rather than "failed". A user who names a ring size with `--ddp` N fixes the number of ranks, and the ring is then never released or shrunk.

Synchronising less often. `--ddp-sync fast` synchronises every H steps instead of every step, in the manner of DiLoCo [19] and post-local SGD [18], on this schedule:

1. Exact synchronisation as a warm-up for a tenth of the run, at least 10 and at most 500 steps (10 when the run's length is unknown).
2. Local steps, after which the ranks' weights are averaged by the number of samples each processed; every epoch ends on a synchronisation.
3. The optimiser's moments are averaged every tenth round.
4. Over the last tenth of the run, the interval shrinks towards one step.

It never adds epochs or changes the batch or the learning rate. The outer step is plain averaging (outer learning rate 1, no momentum). DiLoCo's Nesterov outer step, momentum 0.9 at an outer learning rate of 0.7, settles at $0.7/(1 - 0.9) = 7$ times the averaged change; at intervals of a few hundred steps that is harmless, but at the 2 to 64 steps used here it multiplies the script's learning rate, and it failed the accuracy bar at every interval tried. The bar is one-sided: the final validation loss may be at most 1% above one device's, as a mean over three seeds, on every test model. The interval depends on the ring's size: the largest that passed on every model was 32 steps at two ranks, 16 at three and 4 at four, and none passed at eight ranks, so none is used for five or more (Section 5.6). These intervals are the defaults; an explicit fast ring beyond four ranks runs at $H = 4$ with a warning.

Choosing automatically. By default the lead daemon switches a ring to fast synchronisation once, when every rank has reported, the ring is within the validated size, and the predicted fast step, inflated by 10%, beats the exact split and, for a ring not fixed by `--ddp` N, also the fastest rank alone. Under `--ddp` N the ring size is fixed and the automatic choice is between the two sync modes only. Every rank switches at the same step boundary, rank 0 prints why, and a joining rank adopts the ring's mode; a join that takes an automatic fast ring past the validated size returns it to exact synchronisation at the next epoch boundary. A rank whose local round runs past four times the ring's median step time skips that round's remaining

batches but counts them, so every rank stays on the same step.

Losing and regaining ranks. When a machine is lost, the client names the rank and host, the lead daemon stops waiting for it, and the survivors take new shares at the next epoch boundary. Samples are assigned to ranks by a weighted rendezvous hash [48], which keeps each survivor's share exact and over 95% of its samples in place, where hash-modulo routing would move most of them. A rank whose script fails stops every rank at once. Losing rank 0's machine ends the run, since its daemon holds the synchronisation, and the survivors stop with that reason. A returning machine or a new eligible node joins the ring at the next epoch boundary: it loads the lead's model, optimiser and buffer snapshot through the lead daemon, runs one inert batch for each epoch it missed so that the script's loop variables exist, and the world size and shards grow to include it, moving about $1/n$ of the data. Only scripts that use a data loader can be re-spread.

The first synchronisation is a barrier: every rank must reach the opening weight broadcast within a configurable wait, 5 minutes by default, and ranks still missing are dropped and named. The torch gloo and NCCL backends are not used.

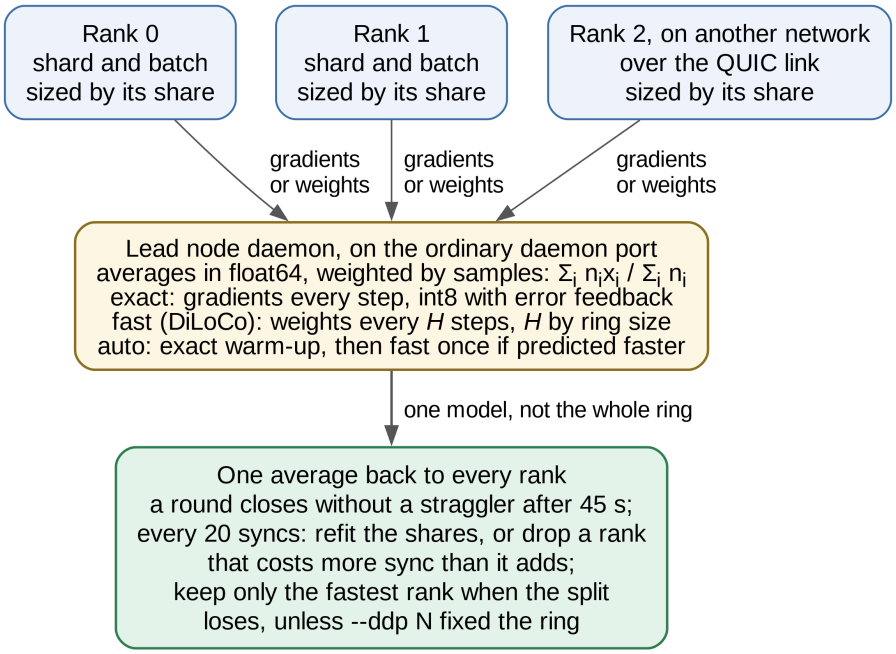


Figure 4.9: Synchronisation over the daemon port. The lead daemon averages by samples and returns one average: 8-bit gradients every step in exact mode, weights every H steps in fast mode; auto switches to fast once.

4.10 The Cost Model

Interception is worthwhile only when the cluster wins, so numerical work is gated. Writing b for the payload in bytes, ϕ for its arithmetic intensity in floating-point operations per byte, ρ for the number of times the data crosses the link, B for link bandwidth and K for the number of nodes that hold the environment, the submitting node included, divisible work is distributed only when

$$\underbrace{\frac{b\phi}{R}}_{\text{compute locally}} > \underbrace{\frac{\rho\, b}{B}}_{\text{move it}} + \underbrace{\frac{b\phi}{R'K} \times 1.3}_{\text{compute remotely}} \tag{4.2}$$

where R is the local throughput, R' the throughput assumed for each of the K nodes, and the factor 1.3 charges 30% overhead against ideal parallel speedup. Matrix multiplication is priced at about 200 GFLOP/s with $\rho = 5$, pandas work at 1 GFLOP/s. Work that cannot be divided — an SVD or eigendecomposition, priced at about 1.5 GFLOP/s and sent whole to one worker — moves when the transfer costs less than half the local computation, since it frees the origin entirely.

The checks run in order. At least two nodes and 32 MiB of payload are required. **If the working set does not fit in local memory, the work goes to the cluster** whatever the transfer costs, and a matrix product the cluster cannot take either raises an error naming the size and the remedy rather than exhausting the machine. If any peer reports a GPU, R' is taken as six times R, a conservative heuristic multiplier rather than a measured speed. The inequality is then evaluated at an optimistic 10 Gbit/s (Figure 4.10); if offloading loses even there, the work stays local without measuring anything. Otherwise bandwidth is *measured*: the daemon's rate over recent real transfers is used when it has one, and otherwise an 8 MiB probe is pushed through the real dispatch path; the result is cached, and a failed probe keeps the work local.

The consequence for dense matrix multiplication is that it stays local on a CPU cluster: local BLAS at about 200 GFLOP/s outruns what a CPU cluster's transport can feed it at practical sizes. Measured against a four-core peer, local won by 25 to 100 times across four shapes, and the model chose local every time. A product ships when it does not fit locally, when a GPU peer makes the model favour it, or when forced. Singular value decomposition, two orders of magnitude slower locally, does ship.

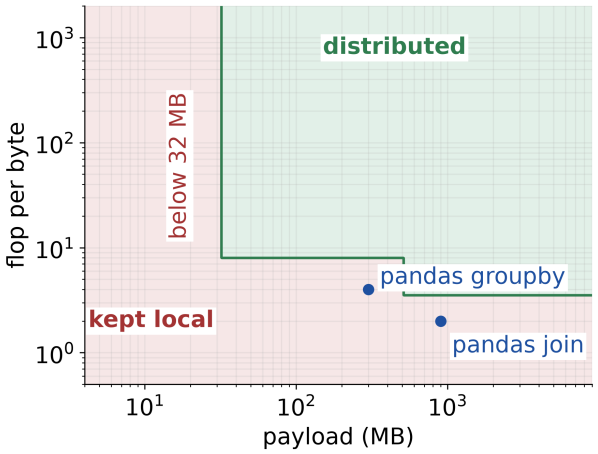


Figure 4.10: Decision regions of Equation 4.2 for the pandas path ($\rho = 1$, $R = R' =$ 1 GFLOP/s) at three nodes on a 500 MB/s link. The step at 512 MiB is a separate rule, not Equation 4.2: work below 8 FLOP per byte stays local up to 512 MiB, so grouped aggregation and joins are distributed only above it.

4.11 Fault Tolerance

The invariant is that using the cluster is never worse than not using it. Five independent layers enforce it for parallel work; a training ring that loses a rank continues on the survivors (Section 4.9.2).

1. **The race.** Chunks are dealt alternately to the local side and the cluster (Figure 4.11), and results fill a slot table first-wins. Idle local cores re-run any chunk the cluster has not yet returned, and the call returns only once every item is covered, so a dead cluster degrades to a plain local pool.

2. **Per-primitive fallback.** Every intercepted call is wrapped, and any error falls back to the original implementation; only a matrix product that fits nowhere raises.

3. **Peer fallback.** A failed peer is skipped for the rest of that chunk and the next candidate tried; if all fail, the work runs locally. A slow peer is raced by a copy of its part.

4. **Worker respawn.** A dead warm worker is restarted, falling back to a one-shot process if that fails.

5. **Rescheduling.** A failed job is placed again with back-off, on an untried node first and at most three times (Section 4.5).

Memory is bounded at the same time: a parallel-map request larger than a node can grant is forwarded to a peer that can, one exceeding 40% of that memory is split into sequential parts, and a request body is capped at the smaller of 2 GiB and half

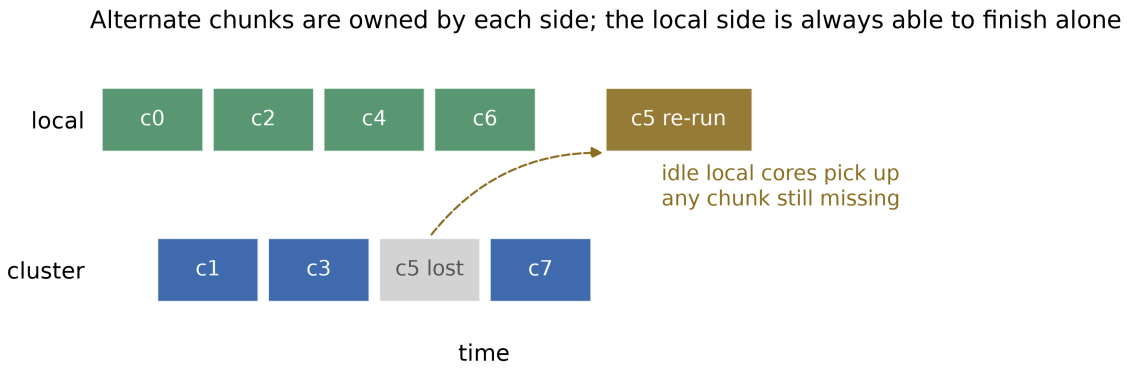


Figure 4.11: Racing local and remote work. The local side is always able to finish alone, which is what makes the guarantee unconditional.

the daemon's memory budget.

4.12 Resource Estimation and Run Reporting

Resource estimation. Tiers are consulted in order, the first that yields a value winning: an explicit `--mem`; 1.2 times the largest peak memory of the recent runs of the same script; or per-library baselines plus 100 MiB for the interpreter. The default when nothing is known is 512 MiB. The worker records each job's peak, which sizes the next run's admission request.

Commands. The CLI has fifteen commands, two of them hidden (`events` and `map`); the visible ones include setup, invite, connect, run, status, attach, doctor and uninstall. Configuration is a single file holding the secret, the introducers and the node's role, and the ports are one block: 4242 for the daemon API and discovery, 4243 for direct links and 4244 for the introducer. `pipedpeer doctor` checks the daemon, the secret, each introducer and each peer (TCP and UDP separately), the firewall, user namespaces, `crun`, Nix, the GPU driver, disk and clock, each with its fix. A plain run behaves like Python: it prints the script's own output, passes the script its arguments and returns its exit code, adding one line on where the work ran when any left the machine.

The run record. Every run keeps one record of timed steps — serialising, sending, queueing, computing and receiving each chunk on each machine, building and sending the environment, each synchronisation round, and the script itself — written by the shim, the daemon and the client, and shown by `-v`, the job history, the web page opened by `pipedpeer ui`, and as CSV or JSON. `pipedpeer events` prints in plain language what this machine's daemon has done, and a dry run reports, before anything runs, the memory estimate, unresolved imports, the parallel calls that can spread, and where the job would go.

Chapter 5

Results

The evaluation asks six questions. Does content-addressed caching make repeated submissions cheap (Section 5.3)? What does interception cost when it decides not to distribute (Section 5.4)? Does parallel work cross machines, and is it shared well across unequal ones (Section 5.5)? Does training across machines stay accurate, and when does it pay (Section 5.6)? Does a run survive the loss of a machine (Section 5.7)? And what happens between real machines on different networks (Section 5.9)?

5.1 Evaluation Setup

Hosts. Laboratory measurements ran on two development hosts: an Intel i5-13420H with 12 hardware threads and 15 GiB of memory under Fedora 43, and a 16-thread laptop with an RTX 3060 laptop GPU under Fedora 44.

Container labs. Worker daemons run in containers on one host. In the *plain lab* (three workers) the containers share the host's processor, memory and loopback interface; it measures correctness, overhead, cache behaviour and fault handling, and cannot measure multi-machine speedup. Where a figure depends on speed or on the network, each container is given a fixed processor and memory limit and the links between containers are shaped by `tc netem` to the latency, bandwidth and loss of a LAN, a Wi-Fi network or a 5G link, with up to 5% loss; speedup is then one container against N containers of the same size. The *LAN lab* reproduces a college laboratory: containers on one bridge network, the host's own daemon as the GPU machine and one container on multi-user Nix, with a two-container variant sharing the host's GPU for training rings.

Two-site testbed. Joining across networks without Tailscale is evaluated on a two-site container testbed on the same host: two containers on separate private networks, both numbered 192.168.0.0/24, each behind a router container whose iptables rules translate addresses and drop unsolicited inbound packets, with the introducer on a third network standing for the internet. Every user command is run across it.

Training accuracy. The accuracy of each synchronisation setting is measured on three models — a four-layer perceptron sized for a GPU (MLP-G), the same

perceptron at a CPU sizing (MLP-C), and a small transformer — against one device trained on the same data and seeds: two-rank rings in the lab, larger rings in-process, with the ranks side by side on a single machine.

Real machines. Two laptops in different homes (RTX 3060 and RTX 3050) connected over a direct Tailscale path [42], a Samsung Galaxy S20 FE phone under Termux, and further measurements between a 20-core machine and a 16-core peer joined over Tailscale, with a public cloud host acting as introducer.

Workloads. Seven example workloads cover a standard-library parallel map, a map over a lambda, scikit-learn with `joblib`, NumPy linear algebra, out-of-core pandas, torch data-parallel training and file synchronisation; one script combines every kind of intercepted work, and five idiomatic, unoptimised user scripts stand in for unseen workloads. Lab wall times are medians of three to five runs, except training rings, which are one seed. Lab accuracy is the mean of three seeds; in-process accuracy is the mean of seeds 0–2 and, where run, of seeds 0–5, and both must pass. Sizes in KiB, MiB and GiB are binary (2^{10}, 2^{20} and 2^{30} bytes); KB, MB and GB are decimal.

5.2 Functional Verification

The test suite exercises the properties the design relies on, with every optional Python library present — among them that a hash-shuffled aggregation equals local pandas, that a parallel map whose every remote part fails still returns the correct result, that the daemon's average is weighted by samples, and that a link hello replayed on another session is refused. A mutation audit guards against a suite that passes without proving anything: it breaks each of 95 features in turn, from the sample-weighted gradient fold to the router-mapping checks, and requires a test to fail. Every mutation was caught.

5.3 Environment Cache

The same trivial job was submitted five times to the plain lab (Figure 5.1). The first submission, which also built the environment, took 23.4 s; the second landed on a worker that did not yet hold the closure and took 8.1 s; once the closure was in place, a submission sent only the workspace and took a median of 0.83 s.

Against a container peer on the submitting laptop, the path-level difference sent 29 MB (7 of 43 store paths) instead of 206 MB when one package was added to an environment the peer held, and a peer holding an environment fetched 0.43 MiB of a 444 MiB closure for one that differed from it by one small pure-Python package.

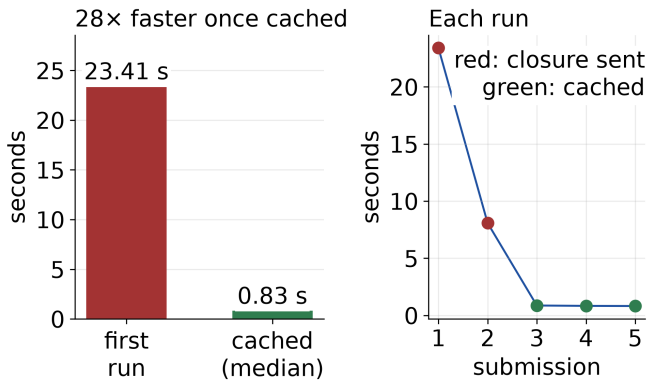


Figure 5.1: Left: first submission against the median of the submissions that sent nothing. Right: every run, coloured by what its log says was sent.

In the LAN lab, a dry-run build of an unseen script's environment took 4 s on a container that had built the common packages ahead of time, against 54 s on one that had not (`docs/benchmarks/rehearsal.md`).

With one of two nodes held to 800 kbit/s, the first run waited the upload's 60 s limit for the slow node and then started; the send to that node continued in the background and held up no run, and once it had finished a run took 2.1 s with the slow node taking a share of the items. With the cache cap lowered, both caches were pruned and the next job still ran on its peer.

5.4 Interception and Sandbox Overheads

A 16-item parallel map whose tail is 0.01 s of work against a 0.5 s dispatch floor is declined by the cost model; a forced run of the same workload does dispatch, so the decline is the model's choice and not a dead cluster. Declining costs about 0.12 s per job, the measurement of the head of the map plus the decision: a median of 0.80 s against 0.68 s with interception disabled. Interpreter startup for a job that imports nothing heavy, best of five runs, is 1.24× that of plain Python when NumPy, pandas, torch and joblib are patched on first import, and 20.0× when they were patched at startup, measured on another host (Figure 5.2). A continuous-integration check fails any change that raises this ratio above 3×.

Sandbox start. The OCI runtime costs 15 to 21 ms more per job than the lighter `bwrap` (averages of 20 iterations, three tasks), small beside a warm job of under a second, and its device model is what makes GPU passthrough possible (Figure 5.3).

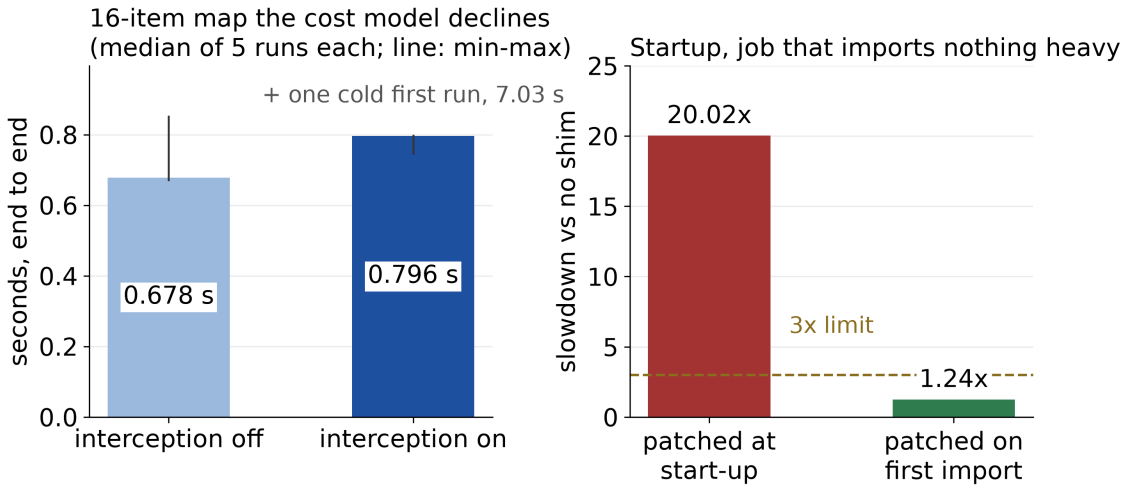


Figure 5.2: Left: the declined 16-item parallel map, median and range; the enabled side's cold first run is left out of its range. Right: interpreter startup with libraries patched at startup and on first import.

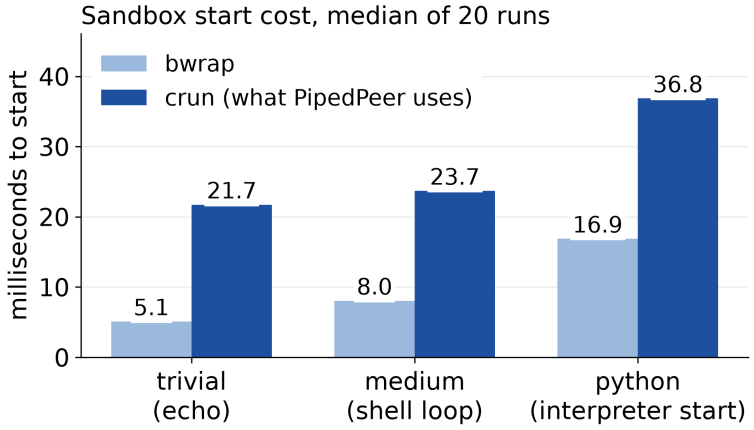


Figure 5.3: Sandbox start cost, median of 20 iterations.

5.5 Distribution Across Machines

Work crosses nodes. With distribution forced, the node running a 48-item parallel map sent 18 items to three other daemons of the plain lab, one of them the submitting machine's own, with results identical to a local run. The NumPy workload moved its 34 MB singular value decomposition to one worker. Because the plain lab's containers share the host's processor, distribution there can only add overhead, and the pool benchmark shows how much: plain Python took 0.17 s, a hand-tuned pool 0.17 s, and the untouched script under PipedPeer 0.24 s with a quarter of the items on the cluster.

Sharing by measured speed. On a lab cluster limited to 8, 2 and 1 CPUs, dividing a map by headcount took 3.6 s (median of five), 1.6 times slower than not

distributing at all. Dividing by measured throughput declined the one-CPU worker and took 2.9 s, faster than the headcount split. Derived from those two figures, it was still about 1.3 times slower than a local run, so at this size the map is better kept on one machine. With one peer held to a tenth of a processor in a cluster of otherwise equal peers, the slowdown against an equal cluster fell from 1.6 times on the first run to 1.08 by the fourth, as the fit learned the slow peer (Table A.2).

Scaling. With each container held to a fixed quota of two processors, a scaling map sweeps one variable at a time with the rest fixed — payload from 1 KB to 64 MB, 8 to 2048 items, per-item cost from 10^3 to 3×10^6 operations, two to four nodes, and a speed spread of up to ten times — and fits each step's growth on a log-log scale from the run records. No CPU step grew faster than its expected order. Four equal nodes ran a map 2.9 times faster than one (0.81 s against 2.37 s): quotas that together stay within the host's processor let this map measure how the cost grows with the node count, which the shared plain lab cannot; it is not a speedup between physical machines.

5.6 Training

One machine or a split. The decision between a ring and one device matters in both directions. In the lab, a small model with heavy steps on a LAN trained in 9.1 s split across two containers of two processors against 11.8 s on one such container, while a 9 MB model with light steps on Wi-Fi took 80.7 s split against 8.0 s on one container. A GPU widens the gap: the training job of Table 5.5, measured in one run on one machine, took 1.5 s on an RTX 3060 against 55.0 s on its processors.

Fast synchronisation. At the default interval for two ranks, 32 steps, in-process runs kept every model within the 1% bar: MLP-G +0.06%, MLP-C +0.64% and the transformer +0.73% over seeds 0–2, and MLP-C +0.35% and the transformer −1.02% over seeds 0–5. Table 5.1 gives one lab run of a two-rank ring at that interval: each rank sent 7.6 to 9.2 times fewer bytes than under exact synchronisation, and the ring finished 1.2–1.4 times faster than exact on the LAN and 1.6–3.1 times on Wi-Fi. A separate lab run at 64 steps, three seeds per cell (Figure 5.4, right), also stayed within the bar on both profiles: MLP-G +0.64%/+0.61%, MLP-C −2.14%/−2.09% and the transformer +0.18%/+0.50% (LAN/Wi-Fi), each within the spread of exact synchronisation's own gap; MLP-C's negative gap reflects that periodic averaging can generalise slightly better than exact synchronisation [18]. The default is 32 rather than 64 because in-process the transformer failed at 64 on seeds 0–2 (+2.13%).

The outer step and the ring size. DiLoCo's outer step (Nesterov momentum 0.9 at an outer learning rate of 0.7) failed the bar at every interval tried, leaving

Table 5.1: Two-rank ring in the lab at the default interval of 32 steps: bytes sent per rank and wall time against one container (one seed)

Model	One device, seconds	MiB per rank, exact / fast	Seconds, exact LAN / Wi-Fi	Seconds, fast LAN / Wi-Fi
MLP-G	27.9	81.1 / 10.7	20.0 / 41.8	15.2 / 18.9
MLP-C	16.9	98.9 / 10.7	14.6 / 43.7	10.4 / 14.1
Transformer	26.7	21.7 / 2.4	22.7 / 35.9	19.1 / 22.9

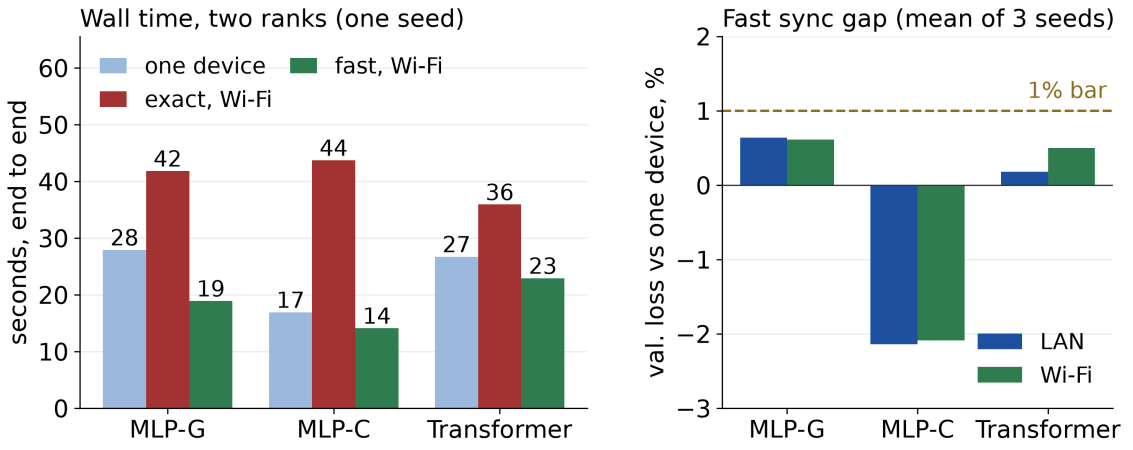


Figure 5.4: Left: wall time of one device and of a two-rank ring under exact and fast synchronisation on Wi-Fi, from the run of Table 5.1. Right: validation-loss gap to one device under fast synchronisation every 64 steps, mean of three seeds, from the separate three-seed lab run; the dashed line is the 1% bar.

the transformer 21.7% above one device even with one synchronisation per epoch (156 local steps), the longest interval these runs allow. In-process runs on the same three models, on up to six seeds, set the interval $H(n)$ as the largest that passed on every model: 32 steps at two ranks, 16 at three and 4 at four; at eight ranks the transformer failed at every interval down to 2, so no interval is used for five or more ranks. Of ten methods tested at 4, 8 and 11 ranks, only exact synchronisation with rank-16 PowerSGD [20] compression and error feedback kept every model within 1% (worst mean +0.18%), at 3.2× fewer bytes than 8-bit exact synchronisation on the perceptrons but 1.7× more on the transformer (Table A.2). Fast synchronisation therefore serves the two to four machines of a small group.

Adapting the ring. Offered ranks of 16, 8 and 2 CPUs, containers with those CPU quotas sharing one host, admission took the 16 and the 8, shared the work 66:34, and ran in 23.6 s against 25.4 s with equal batches. Throttling one rank's container from 8 CPUs to 1 and back moved the shares from 64:36 to 84:16, as the step-time window saw the throttled rank slow down, and then to 70:30 after it recovered.

5.7 Fault Tolerance and Recovery

Table 5.2 lists the failure drills run in the LAN lab and its two-container GPU variant (records in `docs/benchmarks/rehearsal.md`), and the cut link on the two-site testbed. They exercise rescheduling (Section 4.11), re-sharding at an epoch boundary (Section 4.9.2), the cancel frame and the logged output stream. The tests add that a parallel map whose every remote chunk fails still returns the correct result (Section 5.2).

Table 5.2: Failure drills

Drill	Result
A node's link cut during a parallel map, two-site testbed	its **53** items run again on the submitting node once it overran its predicted 18 s; checksum unchanged
A worker killed during a remote job	node reported gone **16** s after the kill
A training rank killed during a run	survivor finished all 80 epochs, **90.5** s
A training rank's machine back during a run	rejoined at the next epoch, **4** s after its daemon started
Interrupt during a parallel map	peers stopped within **2** s
Terminal closed during a run	stopped everywhere, as an interrupt
Detach, then attach while running and after it finished	followed live; replayed in **1.0** s once finished
Submitter's network removed during a remote job	**200** s outage: job ran once, results back **15** s after reconnect
Introducer's address changed, no commands run	recovered in about **12** s

5.8 End-to-End Runs in the LAN Lab

The whole system was run end to end in the LAN lab for the processor steps and its two-container GPU variant for the GPU steps. The containers share one laptop, so Table 5.3 shows what each step costs and that it works, not speedup (records in `docs/benchmarks/rehearsal.md`). A cold run is the first ever on fresh containers; a cached run follows it. The GPU job's first run is not given, because that run measured a closure broadcast to nodes without a GPU rather than the step itself. The last row ran eight jobs at once on containers with 6.0 GB of memory in total.

Table 5.3: End-to-end steps, first run and cached

Step	**Cold**	**Cached**
Parallel map	43.5 s	17.0 s
Map over a lambda	20.5 s	4.8 s
GPU job submitted from a GPU-less node	—	7.6 s
Two-rank GPU ring on one shared GPU	630.3 s	9.5 s
Combined script	664.6 s	26.7 s
Five student-style scripts*	35–126 s each	20–70 s each
Concurrent workloads	412.0 s, all 8 jobs completed	

*First run after the common packages were built ahead of time.

Cold times are dominated by building and fetching environments; the GPU steps download CUDA torch.

5.9 Measurements Between Real Machines

Two laptops in different homes. Table 5.4 gives runs between two laptops connected over a direct Tailscale path with a 6–8 ms round trip. Times are the script's own unless marked from submission; per-run records are in `docs/benchmarks/real-hardware.md`.

Table 5.4: Two laptops over the internet (RTX 3060 and RTX 3050)

Run	**Result**
CPU script (pool, joblib, NumPy, pandas)	about 14 s split, half the items on the peer, against 18 s on one laptop
Whole job sent to the peer	9.1 s; 11.7 s from submission
Small training model, one GPU	4.2 s, validation loss 0.0794
Same, `--ddp 2`, exact synchronisation	37.1 s, loss 0.0797
Same, `--ddp 2`, fast synchronisation	7.4 s, loss 0.0796
Second session: `--ddp 2`, fast synchronisation	8.5 s, loss 0.0797
Second session: `--ddp 2`, automatic choice	14.6 s, fast after a short exact warm-up; loss 0.0796
Combined script, from submission: first run and repeat	506 s (487 s environment build, CUDA torch download); 17.7 s, of which the script 15.6 s

The mixed CPU script ran faster across the two laptops than on one. The small training model did not: one GPU finished first. The rings were fixed with `--ddp 2`,

so the automatic choice could pick only the sync mode, not release a rank; without `--ddp` N the lead releases a ring that one rank alone outruns. The first four rows come from one session, in which fast synchronisation ran 5.0 times faster than exact (7.4 against 37.1 s) at about the same loss. The last two come from a second session on the build whose automatic choice can switch a fixed ring to fast mode; exact synchronisation was not rerun there. In it the automatic choice, which starts exact and switches once, took 14.6 s against 8.5 s for fast.

Across two NATs. On the two-site testbed of Section 5.1, scripts, parallel maps, remote runs and both training sync modes ran across the sites, and a node whose link was cut mid-run was replaced. Two machines meet directly when one introducer is reachable; otherwise a public address or an overlay such as Tailscale is required. The two laptops above ran over Tailscale for that reason: neither home offered a reachable introducer (one router refuses port mapping, the other machine sits behind two NATs). No relay is involved in either case.

A phone. On a Samsung Galaxy S20 FE (Android 13, Termux), a laptop's CPU script spread over that laptop, a second laptop over the internet and the phone gave the phone 8 of 192 parallel items with an unchanged result, after the phone had built the job's environment from the binary cache in 27 s. A torch script training 160 small models gave the phone 6 of them, all converged; with 32, it took none.

Further measurements. Table 5.5 collects measurements between a 20-core machine, a 16-core peer and a public cloud host, each taken a few times.

Table 5.5: Further measurements between real machines

Measurement	**Result**
Parallel map, 20-core machine and 16-core peer over Tailscale, plain Python against PipedPeer	5.8–6.4 s against 4.9–5.4 s; 116 of 256 items on the peer
Training over a home link of about 54 Mbit/s: one machine, two ranks with 16-bit gradients, two ranks with 8-bit gradients	54.9 / 69.4 / 56.1 s; loss 0.0875 / 0.0875 / 0.0877
Introducer traffic during a cross-network run	0.2 MB, against about 30 MB between the peers
Introducer suspended during a run	live peer still reached in 0.03 s
Signed closure on a host that requires signatures	imported when the sender's key was trusted; refused when it was removed
Daemon memory ceiling, 3 GiB allocation under a 1 GiB ceiling	child killed in seconds, out-of-memory recorded

A parallel map ran faster across two machines than on one, though short of what the core counts allow, since a job of about 5 s is dominated by setup and serialisation;

exact training over a home link did not beat one machine; the introducer carried a negligible share of the traffic.

5.10 Comparative Analysis

Table 5.6: PipedPeer against existing systems. The lower block is where PipedPeer is partial or the weaker system.

Property	**PipedPeer**	**Spark**	**Ray**	**Dask**	**BOINC**	**ssh**
What it does differently						
Runs unmodified Python	yes	no	partial	partial	no	yes
No cluster to configure	yes	no	no	no	no	partial
No head or master node	yes	no	no	no	no	yes
Reproduces the environment	yes	partial	partial	partial	partial	no
Joins NATed machines, no relay[†]	yes	no	no	no	no	no
Where it is partial or behind						
Survives losing a node[*]	partial	yes	yes	yes	yes	no
Authenticated, encrypted	partial	yes	partial	partial	yes	yes
Sandboxes each job	partial	no	no	no	partial	no
Runtimes beyond Python	no	yes	partial	no	yes	yes
Proven in production	no	yes	yes	yes	yes	yes

[†]When one introducer is reachable; otherwise a public address or an overlay such as Tailscale is needed.
[*]Any worker; not the submitter of a parallel script or the machine of a training ring's rank 0.

Table 5.6 sets PipedPeer against Spark, Ray, Dask, BOINC and plain `ssh`. The upper block lists the criteria on which PipedPeer was designed to differ, so it meets them by design; the lower block is the more informative half (Section 4.3). Spark, Ray and Dask are far more capable at scale, with mature schedulers and tooling; PipedPeer trades maturity and scale for the elimination of setup, code change and network configuration. BOINC [6], closest in spirit, is a platform for specific projects with their own servers rather than a general executor. No benchmark was run against these systems; their entries come from their published designs.

Chapter 6

Conclusion

6.1 Conclusion

PipedPeer runs an unmodified Python script on other machines with no cluster to configure, reproduces its environment exactly, distributes the parallel work inside it without the user touching the source, and joins machines on different networks with a single command and no relay. It ships environments as signed Nix closures, sending a peer only the store paths it lacks. It substitutes the parallel primitives a script already uses and divides their work by measured speed, under a cost model that keeps work local when moving it would be slower. It joins machines on different networks through an introducer that every daemon serves and that carries no job data. It places each job from the submitting client, under a three-phase lease, with no scheduler process. It keeps the result of every distributed path by falling back to a local one. It trains PyTorch models data-parallel through the daemons, weighting batches by measured speed and choosing exact or fast synchronisation by ring size. In the evaluation, a worker that held an environment started a job in 0.83 s against 23.4 s cold, one that held part of it received 86% fewer bytes, and at 32 steps on two ranks fast synchronisation held accuracy while each rank sent 7.6 to 9.2 times fewer bytes.

The principle that runs through the design is that the cluster is used only when a measurement says it will help: work is divided by measured speed rather than by headcount, dense matrix products stay local on a CPU cluster, a training ring whose size the user did not fix is released when one GPU is faster, and fast synchronisation is chosen only for ring sizes where it kept accuracy.

6.2 Limitations

- **No relay.** Two machines between which no direct path exists — for example two carrier networks that both allocate a new port per destination — cannot work together; such a peer is reported and routed around.
- **Python and Linux only**, with NVIDIA GPUs in practice; phones take parallel chunks only.

- **Submitter and lead.** A parallel script runs on its submitter, which must survive the job, and a training ring needs rank 0's machine.
- **Fast synchronisation for small rings.** No interval kept accuracy at eight ranks, so automatic mode keeps rings of five or more exact.
- **Interception gaps.** Only `map` of a process-pool executor is intercepted, the `@` operator is not, and numerical work under 32 MiB never leaves the node, by design.
- **A partial sandbox.** The job runs as user 0 inside the container with the host's network and no seccomp filter, and isolation is skipped with a warning on a node lacking `crun` or user namespaces.
- **Measurement.** Speedup comes from containers with fixed processor quotas and a few real machines; the success rate of hole punching across many networks, two GPUs in one machine and fast synchronisation at five to seven ranks are not measured. NAT traversal is evaluated on the two-site testbed on one host and over an overlay between real homes.
- **Seed spread.** The 1% accuracy bar applies to the mean over seeds; single seeds spread wider, from -3.9% (the lab run at 64 steps) to $+2.2\%$ (in-process at 32 steps) against one device, so one run can miss the bar while the mean meets it.
- **Trust.** Every holder of the cluster secret can run code on every node (Section 4.3).

Chapter 7

Future Scope

The most pressing work is **closing the security gaps**: binding trusted signing keys to the identities that authenticated a link, requiring TLS rather than preferring it, checking signatures on every import path, and authorising users more finely than by one shared secret. Next to it, **a relay of last resort** would join the peer pairs that no direct path can reach, without making it the default data path.

Placement would gain from **locality and topology**: accounting for which node already holds the input data, as iDDS [25] does, and for network distance through a graph-structured view of the cluster [31]. **Fault tolerance for the submitter and the lead** would let a parallel script outlive its submitter and a training ring outlive rank 0's machine, and **a cluster-wide view** would merge every machine's events in one place.

The system can reach more hardware and more programs. **More accelerators** means AMD GPUs through ROCm and phones that run whole jobs; **memory escalation** on out-of-memory would try gradient accumulation, checkpointing and lower precision before failing; and **additional runtimes** would extend interception beyond Python.

For training, **fast synchronisation for larger rings** could build on compressed exact synchronisation with PowerSGD, which kept accuracy at 4 to 11 ranks in this work, and on synchronising the model in staggered fragments (Streaming DiLoCo [49]). Finally, **a controlled multi-machine evaluation** across separate machines and networks, including a machine with two GPUs, would measure speedup where the container lab cannot.

Chapter 8

Research Publication

8.1 PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination

A research paper drawn from this work, "PipedPeer: Zero-Configuration Distributed Computing via Recursive, Decentralised Coordination", is written in the IEEE conference format by Yateen Vaviya, Vinayak Yadav and Sahil Gupta, with Dr. Rupali Sawant and Prof. Dayanand Ambawade of the Department of Computer Engineering, Sardar Patel Institute of Technology. Status: .

The paper presents PipedPeer for a systems readership and is organised as follows.

- **Abstract and Introduction** state the three obstacles to using nearby idle machines — the environment, the code and the network — and the paper's contributions.
- **Related Work** places PipedPeer against cluster frameworks for Python, volunteer computing, environment reproduction, NAT traversal, and communication-efficient data-parallel training.
- **Design** sets out the design goals and the architecture of Chapter 3: one daemon per machine, coordination by the submitting machine, and an introducer in every daemon.
- **Implementation** summarises the mechanisms of Chapter 4: content-addressed environments and path-level transfer, transparent interception with measured work sharing and the cost model, training synchronised through the daemon, and joining across networks.
- **Evaluation** states its questions and answers them with the measurements of Chapter 5, in the order of setup, microbenchmarks, end-to-end runs and training.
- **Limitations and Conclusion** close the paper.

Bibliography

[1] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," in *Proc. 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*, 2010.

[2] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica, "Ray: A distributed framework for emerging AI applications," in *Proc. 13th USENIX OSDI*, 2018, pp. 561–577.

[3] M. Rocklin, "Dask: Parallel computation with blocked algorithms and task scheduling," in *Proc. 14th Python in Science Conference (SciPy)*, 2015, pp. 126–132.

[4] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. 6th USENIX OSDI*, 2004, pp. 137–150.

[5] D. Petersohn et al., "Towards scalable dataframe systems," *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 2033–2046, 2020.

[6] D. P. Anderson, "BOINC: A system for public-resource computing and storage," in *Proc. 5th IEEE/ACM Workshop on Grid Computing*, 2004, pp. 4–10.

[7] E. Dolstra, M. de Jonge, and E. Visser, "Nix: A safe and policy-free system for software deployment," in *Proc. 18th LISA*, 2004, pp. 79–92.

[8] Open Container Initiative, "OCI Runtime Specification, v1.0.2," 2020. [Online]. Available: https://github.com/opencontainers/runtime-spec

[9] crun, "A fast and lightweight fully featured OCI runtime and C library for running containers," 2026. [Online]. Available: https://github.com/containers/crun. Accessed: Oct. 16, 2026.

[10] S. Sekigawa, C. Sasaki, and A. Tagami, "Web application-based WebAssembly container platform for extreme edge computing," in *Proc. IEEE GLOBECOM*, 2023, pp. 3609–3614.

[11] M. N. Hoque and K. A. Harras, "WebAssembly for edge computing: Potential and challenges," *IEEE Commun. Standards Mag.*, vol. 6, no. 4, pp. 68–73, Dec. 2022.

[12] S. Kakati and M. Brorsson, "A cross-architecture evaluation of WebAssembly in the cloud-edge continuum," in *Proc. IEEE/ACM CCGrid*, 2024, pp. 337–346.

[13] J. Bachmeier, V. Yussupov, J. Henß, and H. Koziolek, "WebAssembly with wasi-nn for edge machine learning inference: Experiences and lessons learned," in *Proc. ECSA*, 2025, pp. 343–359.

[14] S. E. Khelifa, M. Bagaa, A. O. Messaoud, and A. Ksentini, "Case study of WebAssembly runtimes for AI applications on the edge," in *Proc. Global Information Infrastructure and Networking Symp. (GIIS)*, 2024, pp. 1–6.

[15] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, "PyTorch distributed: Experiences on accelerating data parallel training," *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 3005–3018, 2020.

[16] T. Dettmers, "8-bit approximations for parallelism in deep learning," in *Proc. 4th Int. Conf. Learning Representations (ICLR)*, 2016; arXiv:1511.04561.

[17] F. Seide, H. Fu, J. Droppo, G. Li, and D. Yu, "1-bit stochastic gradient descent and its application to data-parallel distributed training of speech DNNs," in *Proc. Interspeech*, 2014, pp. 1058–1062.

[18] T. Lin, S. U. Stich, K. K. Patel, and M. Jaggi, "Don't use large mini-batches, use local SGD," in *Proc. 8th Int. Conf. Learning Representations (ICLR)*, 2020;

arXiv:1808.07217.

[19] A. Douillard, Q. Feng, A. A. Rusu, R. Chhaparia, Y. Donchev, A. Kuncoro, M. Ranzato, A. Szlam, and J. Shen, "DiLoCo: Distributed low-communication training of language models," arXiv:2311.08105, 2023.

[20] T. Vogels, S. P. Karimireddy, and M. Jaggi, "PowerSGD: Practical low-rank gradient compression for distributed optimization," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019.

[21] Y. Li and S. Fujita, "Design of Elixir-based edge server for responsive IoT applications," in *Proc. 10th Int. Symp. Computing and Networking Workshops (CANDARW)*, 2022, pp. 185–191.

[22] Y. Li and S. Fujita, "A synergistic Elixir-EDA-MQTT framework for advanced smart transportation systems," *Future Internet*, vol. 16, no. 3, Art. no. 81, 2024.

[23] J. Spenger, "Programming models for failure-transparent distributed systems," Ph.D. dissertation, KTH Royal Institute of Technology, Stockholm, Sweden, 2025. [Online]. Available: https://www.diva-portal.org/smash/get/diva2:2013047/FULLTEXT01.pdf

[24] A. Meurer, A. Reines, R. Gommers, Y.-L. Fang, J. Kirkham, M. Barber, S. Hoyer et al., "Python array API standard: Toward array interoperability in the scientific Python ecosystem," in *Proc. 22nd Python in Science Conf. (SciPy)*, 2023, pp. 8–17. [Online]. Available: https://proceedings.scipy.org/articles/gerudo-f2bc6f59-001

[25] W. Guan, T. Maeno, A. Alekseev, F. H. Barreiro Megino, K. De, E. Karavakis, A. Klimentov et al., "iDDS: Intelligent distributed dispatch and scheduling for workflow orchestration," arXiv:2510.02930, 2025. [Online]. Available: https://arxiv.org/abs/2510.02930

[26] Y. Deng, Z. Chen, X. Chen, and Y. Fang, "Task offloading in multi-hop relay-aided multi-access edge computing," *IEEE Trans. Veh. Technol.*, vol. 72, no. 1, pp. 1372–1376, Jan. 2023.

[27] H. He, X. Yang, X. Mi, H. Shen, and X. Liao, "Multi-agent deep reinforcement learning based dynamic task offloading in a device-to-device mobile-edge computing network to minimize average task delay with deadline constraints," *Sensors*, vol. 24, no. 16, Art. no. 5141, 2024.

[28] D. Yong, R. Liu, X. Jia, and Y. Gu, "Joint optimization of multi-user partial offloading strategy and resource allocation strategy in D2D-enabled MEC," *Sensors*, vol. 23, no. 5, Art. no. 2565, 2023.

[29] S. Wang, S. Zhao, H. Gui, X. He, Z. Lu, B. Chen, Z. Fan, and S. Pang, "Energy-efficient collaborative task offloading in multi-access edge computing based on deep reinforcement learning," *Ad Hoc Networks*, vol. 169, Art. no. 103743, Mar. 2025.

[30] C. Li, Q. Chen, M. Chen, Z. Su, Y. Ding, D. Lan, and A. Taherkordi, "Blockchain enabled task offloading based on edge cooperation in the digital twin vehicular edge network," *J. Cloud Comput.*, vol. 12, Art. no. 120, 2023.

[31] Z. Zhao, J. Perazzone, G. Verma, and S. Segarra, "Congestion-aware distributed task offloading in wireless multi-hop networks using graph neural networks," in *Proc. IEEE ICASSP*, 2024, pp. 8951–8955.

[32] M. Petit-Huguenin, G. Salgueiro, J. Rosenberg, D. Wing, R. Mahy, and P. Matthews, "Session Traversal Utilities for NAT (STUN)," RFC 8489, Internet Engineering Task Force, Feb. 2020.

[33] D. MacDonald and B. Lowekamp, "NAT behavior discovery using Session Traversal Utilities for NAT (STUN)," RFC 5780, Internet Engineering Task Force, May 2010.

[34] S. Cheshire and M. Krochmal, "NAT Port Mapping Protocol (NAT-PMP),"

RFC 6886, Internet Engineering Task Force, Apr. 2013.

[35] D. Wing, S. Cheshire, M. Boucadair, R. Penno, and P. Selkirk, "Port Control Protocol (PCP)," RFC 6887, Internet Engineering Task Force, Apr. 2013.

[36] UPnP Forum, "Internet Gateway Device (IGD) V2.0," Standardized DCP, 2010.

[37] B. Ford, P. Srisuresh, and D. Kegel, "Peer-to-peer communication across network address translators," in *Proc. USENIX Annual Technical Conf.*, 2005, pp. 179–192.

[38] J. Iyengar and M. Thomson, "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, Internet Engineering Task Force, May 2021.

[39] E. Rescorla, "The Transport Layer Security (TLS) protocol version 1.3," RFC 8446, Internet Engineering Task Force, Aug. 2018.

[40] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," *J. Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.

[41] Y. Collet and M. Kucherawy, "Zstandard compression and the 'application/zstd' media type," RFC 8878, Internet Engineering Task Force, Feb. 2021.

[42] D. Anderson, "How NAT traversal works," Tailscale blog, Aug. 2020. [Online]. Available: https://tailscale.com/blog/how-nat-traversal-works. Accessed: Oct. 16, 2026.

[43] E. Rescorla, "Keying material exporters for Transport Layer Security (TLS)," RFC 5705, Internet Engineering Task Force, Mar. 2010.

[44] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, Internet Engineering Task Force, Feb. 1997.

[45] cloudpickle, "Extended pickling support for Python objects," 2026. [Online]. Available: https://github.com/cloudpipe/cloudpickle. Accessed: Oct. 16, 2026.

[46] Termux, "Termux: Android terminal emulator and Linux environment," 2026. [Online]. Available: https://termux.dev. Accessed: Oct. 16, 2026.

[47] PRoot, "chroot, mount `--bind`, and binfmt_misc without privilege/setup for Linux," 2026. [Online]. Available: https://proot-me.github.io. Accessed: Oct. 16, 2026.

[48] D. G. Thaler and C. V. Ravishankar, "Using name-based mappings to increase hit rates," *IEEE/ACM Trans. Networking*, vol. 6, no. 1, pp. 1–14, Feb. 1998.

[49] A. Douillard, Y. Donchev, K. Rush, S. Kale, Z. Charles, Z. Garrett, G. Teston, D. Lacey, R. McIlroy, J. Shen, A. Ramé, A. Szlam, M. Ranzato, and P. Barham, "Streaming DiLoCo with overlapping communication: Towards a distributed free lunch," arXiv:2501.18512, 2025.

[50] P. Goyal, P. Dollár, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He, "Accurate, large minibatch SGD: Training ImageNet in 1 hour," arXiv:1706.02677, 2017.

[51] E. Hoffer, I. Hubara, and D. Soudry, "Train longer, generalize better: Closing the generalization gap in large batch training of neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 1729–1739.

[52] T. B. Johnson, P. Agrawal, H. Gu, and C. Guestrin, "AdaScale SGD: A user-friendly algorithm for distributed training," in *Proc. 37th Int. Conf. Machine Learning (ICML)*, PMLR vol. 119, 2020, pp. 4911–4920.

[53] A. Sarfi, B. Thérien, J. Lidin, and E. Belilovsky, "Overcoming the communication-performance tradeoff in LLM pretraining," arXiv:2508.15706, 2025.

[54] A. F. Aji and K. Heafield, "Sparse communication for distributed gradient descent," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2017, pp. 440–445.

[55] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Machine Learning and Systems (MLSys)*, vol. 2, 2020, pp. 429–450.

Appendix A

Implementation Size and Further Measurements

A.1 Scale of the Implementation

Table A.1 gives the size of the implementation and of its test suite.

Table A.1: Implementation size

Measure	Value
Go source, excluding tests*	49,576 lines across 191 files
Go tests*	38,114 lines across 216 files
Python interception shim, embedded	7,251 lines
Go packages	47: the binary's 46 (45 internal and `main`) and the separate benchmark module's one
Mutations in the test audit	95, all caught

*Every `.go` file in the source tree at the final revision, split only by the `_test.go` suffix; lines counted with `wc -l`, blank and comment lines included.

A.2 Further Measurements

Table A.2 lists measurements that support individual mechanisms of Chapter 4 but are not needed for the questions of Chapter 5. All are from the container labs of Section 5.1 unless stated.

Table A.2: Further measurements

Measurement	Result
Parallel-map request body for a 100 MB payload	built in 0.06 s, for 100 items of 1 MB and for 1000 items
Workspace re-upload after editing one file (40 files of 200 KB, two-rank run on two nodes, LAN)	8.0 MB stored on the second node on the first run, 200 KB after the edit
Empty script, environment already on the node	0.23 s total overhead

Measurement	**Result**
Many-rank remedies for fast synchronisation, in-process, at 4, 8 and 11 ranks	rank-16 PowerSGD with error feedback on exact synchronisation within 1% on every model (worst mean +0.18%), sending 3.2× fewer bytes than 8-bit exact synchronisation on the perceptrons and 1.7× more on the transformer; keeping each rank's batch, with or without learning-rate scaling [50–52] or doubled epochs, SparseLoCo [53], top-k [54], FedProx [55], rank-4 PowerSGD and outer learning rates of 1.25 or more all failed
One peer at a tenth of a processor among equal peers (lab), slowdown against an equal cluster on four consecutive runs	1.60, 1.25, 1.09 and 1.08 times; equal clusters unchanged
Skipping a stalled rank's remaining batches, by a factor of 2 instead of 4 times the median step, in a simulation of the ring	28–30% less time for 0.3–1.2% more loss, so the default only guards against a stalled rank
Phone (S20 FE): the CPU script without its pandas step, three runs each	62.4, 62.8 and 66.0 s with Nix's Python under `proot`; 73.1, 80.8 and 72.8 s with Termux's own Python (`docs/benchmarks/real-hardware.md`)
Phone runtime closure (Python with NumPy, pandas, torch, scikit-learn and joblib, ARM)	67 paths, about 1.5 GiB unpacked and 395 MiB to download, all from the public binary cache
Two daemons on one machine, a host daemon and a container worker	memory promised in total 6.74 GiB and 1.97 GiB, each daemon subtracting the other's reservations
A joining machine on the two-site testbed, time to connect	0.2–0.4 s